ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

NGÀNH: CÔNG NGHỆ THÔNG TIN

GVHD: ThS. Phạm Thi Vương

Nhóm: 21

SVTH:	Nguyễn Công Quân	3123560070
	Nguyễn Quốc Đại	3123560013
	Nguyễn Thái Vinh	3123410433

Thành phố Hồ Chí Minh, năm 2026

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

NGÀNH: CÔNG NGHỆ THÔNG TIN

GVHD: ThS. Phạm Thi Vương

Nhóm: 21

SVTH:	Nguyễn Công Quân	3123560070
	Nguyễn Quốc Đại	3123560013
	Nguyễn Thái Vinh	3123410433

Thành phố Hồ Chí Minh, năm 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng đồ án môn học *"Tìm hiểu công nghệ Next.js qua Ứng dụng tra cứu & khám phá phim ảnh"* là công trình nghiên cứu và phát triển độc lập của nhóm sinh viên thực hiện dưới sự hướng dẫn chuyên môn của ThS. Phạm Thi Vương.

Các kết quả kiểm chứng trong báo cáo được ghi theo mã nguồn và lệnh đã chạy. Nhóm không công bố số đo Core Web Vitals, kích thước bundle hoặc Streaming TTFB khi chưa có dữ liệu gốc. Tài liệu và nguồn dữ liệu tham khảo được liệt kê ở cuối báo cáo.

Nhóm xin chịu hoàn toàn trách nhiệm trước Khoa Công nghệ Thông tin và Trường Đại học Sài Gòn về tính trung thực và liêm chính học thuật của đồ án này.

Thành phố Hồ Chí Minh, ngày 03 tháng 10 năm 2026

Đại diện nhóm sinh viên thực hiện

Nguyễn Công Quân **Nguyễn Quốc Đại** **Nguyễn Thái Vinh**

LỜI CẢM ƠN

Để hoàn thành đồ án môn học này một cách chỉn chu và đạt được chiều sâu kỹ thuật cao, nhóm chúng em đã nhận được sự quan tâm, tạo điều kiện và hướng dẫn quý báu từ quý Thầy, Cô và các tổ chức liên quan.

Trước hết, nhóm xin gửi lời tri ân chân thành và sâu sắc nhất đến **ThS. Phạm Thi Vương** – Giảng viên phụ trách học phần Các Công nghệ Lập trình Hiện đại. Thầy không chỉ truyền đạt những kiến thức nền tảng vững chắc và định hướng tiếp cận công nghệ đúng đắn: *"Trọng tâm của môn học là làm chủ CÔNG NGHỆ, không phải sa đà vào nghiệp vụ phức tạp; ứng dụng xây dựng chỉ là phương tiện để áp dụng và minh họa"*, mà còn luôn nhiệt tình theo sát, giải đáp các vướng mắc kỹ thuật hóc búa và định hướng nhóm giải quyết bài toán cốt lõi.

Nhóm cũng xin chân thành cảm ơn Ban Chủ nhiệm Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn đã xây dựng chương trình đào tạo hiện đại, tạo lập môi trường học tập thực tế giúp sinh viên có cơ hội tiếp cận và làm chủ những công nghệ mới nhất trên thế giới.

Cuối cùng, nhóm xin gửi lời cảm ơn đến cộng đồng mã nguồn mở Next.js (Vercel), React Core Team và nền tảng The Movie Database (TMDB) đã cung cấp các tài liệu kỹ thuật chuẩn xác và nguồn dữ liệu mở chất lượng cao để nhóm hoàn thiện đồ án này.

Dù đã rất nỗ lực nghiên cứu và hoàn thiện, song do thời gian có hạn và kiến thức thực tế còn đang trong quá trình tích lũy, đồ án chắc chắn không tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những lời nhận xét, góp ý quý báu của Thầy Cô để sản phẩm ngày càng hoàn thiện hơn.

Nhóm sinh viên thực hiện

TÓM TẮT ĐỒ ÁN

Báo cáo nghiên cứu Next.js 16 (App Router) qua ứng dụng MovieHub sử dụng React 19, TypeScript và dữ liệu phim từ TMDB API. Trọng tâm là mô hình kết xuất của Next.js, ranh giới Server/Client Component, dữ liệu phía server, định tuyến, cache và Streaming với React Suspense; nghiệp vụ tra cứu phim chỉ là bối cảnh minh họa.

Trang chủ là Server Component, lấy bốn nhóm dữ liệu song song bằng `Promise.all`. Các thẻ `MovieCard` được tạo ở server và truyền dưới dạng phần tử React cho `MovieRow` phía client để thanh cuộn xử lý sự kiện mà không nhập trực tiếp mã của thẻ phim vào client bundle. Trang `/movie/[id]` dùng tham số bất đồng bộ, metadata động và hai ranh giới Suspense cho diễn viên và phim tương tự. Trang `/browse` và `/search` lưu bộ lọc trên URL. Watchlist dùng Server Action ghi cookie `HttpOnly`; tìm kiếm gợi ý dùng Route Handler GET cùng debounce và `AbortController`.

Bằng chứng hiện có gồm mã nguồn, kiểm thử tự động, kết quả lint/type-check và bảng phân loại route của bản build production. Báo cáo không khẳng định điểm Lighthouse, kích thước bundle hay mức cải thiện TTFB khi chưa có dữ liệu đo gốc và kịch bản đối chứng có thể tái lập. Các giới hạn này được nêu rõ tại Chương 4.

Từ khóa: *Next.js 16, App Router, React Server Components, Streaming, Suspense, ISR, Server Actions, Route Handler, TMDB.*

MỤC LỤC

LỜI CAM ĐOAN **i**

LỜI CẢM ƠN **ii**

TÓM TẮT ĐỒ ÁN **iii**

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ **viii**

DANH MỤC BẢNG BIỂU **x**

DANH MỤC HÌNH ẢNH **xi**

LỜI MỞ ĐẦU **1**

1 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG **4**

1.1. Tổng quan về Next.js và lịch sử phát triển 4

1.1.1. Bối cảnh ra đời và hạn chế của React Single Page Application (SPA) 4

1.1.2. Bước chuyển đổi với Pages Router (Next.js 1 – 12) 4

1.1.3. Kỷ nguyên mới với App Router và Server-Driven UI (Next.js 13 – 16) 5

1.1.4. So sánh Next.js App Router với các giải pháp hiện đại khác . 6

1.1.5. Phản biện kỹ thuật: Khi nào KHÔNG nên sử dụng Next.js? . 7

1.2. Kiến trúc React Server Components (RSC) và Client Components . . 9

1.2.1. Bản chất và nguyên lý hoạt động của RSC 9

1.2.2. Định dạng dữ liệu RSC Payload và cây render hỗn hợp 9

1.2.3. Chỉ thị `"use client"` và quy tắc Composition Pattern . . 10

1.3. Cơ chế Caching và Data Fetching đa tầng trong Next.js 16 12

1.3.1. Tính hợp lệ của API `fetch` và tùy chọn `revalidate` . . . 12

1.3.2. Cơ chế Cache Components trong Next.js 16 13

1.3.3. Tác động của việc đọc `cookies()` đến quá trình Render và Caching 14

1.3.4. Mối quan hệ tương hỗ giữa Streaming Suspense và Data Caching 14

1.3.5. Các cơ chế nền tảng bổ trợ: Hydration, Vòng đời Hook và URL State 14
1.4. Kỹ thuật Streaming UI và React Suspense 15
1.4.1. Phạm vi chặn và streaming trong MovieHub 15
1.5. Cơ chế Server Actions và xử lý Mutation dữ liệu 16
1.5.1. Kiến trúc Remote Procedure Call (RPC) 16
1.5.2. Bảo mật dữ liệu, kiểm thực đầu vào và giới hạn Cookie HttpOnly 17
1.6. Hệ sinh thái công nghệ bổ trợ 17
1.6.1. Thư viện React 19 17
1.6.2. Framework giao diện Tailwind CSS v4 17
1.6.3. Ngôn ngữ lập trình TypeScript 18
1.7. Nền tảng dữ liệu mở The Movie Database (TMDB) API 18

2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 19
2.1. Khảo sát và đặc tả yêu cầu hệ thống 19
2.1.1. Yêu cầu chức năng (Functional Requirements) 19
2.1.2. Yêu cầu phi chức năng (Non-Functional Requirements) . . . 21
2.2. Kiến trúc hệ thống tổng thể 21
2.2.1. Mô hình kiến trúc 3 tầng phân lớp 21
2.2.2. Cấu trúc thư mục dự án theo chuẩn App Router 22
2.3. Thiết kế tích hợp dịch vụ và luồng dữ liệu 23
2.3.1. Kiến trúc tầng dịch vụ `services/client.ts` 23
2.3.2. Xử lý lỗi mạng và phạm vi DNS 24
2.3.3. Phân tầng chiến lược Caching và Revalidation 24
2.4. Thiết kế giao diện và trải nghiệm người dùng (UI/UX) 25
2.4.1. Hệ thống thiết kế chuẩn điện ảnh (Netflix Dark Cinema Design System) 25
2.4.2. Sơ đồ điều hướng và luồng trải nghiệm người dùng 25
2.5. Môi trường và công cụ phát triển 26

3 HIỆN THỰC HỆ THỐNG 27
3.1. Cài đặt và cấu hình dự án 27
3.1.1. Mô hình dữ liệu và phần không áp dụng 28
3.2. Hiện thực Tầng 1A: Các tính năng cốt lõi bắt buộc 29
3.2.1. Phân định ranh giới Server/Client Component 29

3.2.2. Server-side Data Fetching và song song hóa với `Promise.all` 29
3.2.3. Dynamic Routing (`/movie/[id]`) và Dynamic SEO Metadata 30
3.3. Hiện thực Tầng 1B: Các kỹ thuật chuyên sâu và nâng cao 31
3.3.1. Server Action cho mutation và Route Handler cho tìm kiếm . 31
3.3.2. Streaming Suspense và Skeleton Loading States 31
3.3.3. Tối ưu hóa hình ảnh với `next/image` và YouTube Player API 32
3.4. Hiện thực các module giao diện người dùng chính 32
3.4.1. Module Hero Banner & Trailer Modal xem trailer tức thì . . . 32
3.4.2. Module MovieRow Slider & Carousel cuộn vô tận 32
3.4.3. Module FilterBar & Trang Khám phá `/browse` 32
3.4.4. Module Thanh điều hướng nâng cao (`NavbarDropdown`, `NavSearchBar`) 33
3.4.5. Module Khám phá Diễn viên và Tuyển tập Phim tương tự . . 33
3.5. Hiện thực cơ chế tự phục hồi và khử trùng lặp dữ liệu 34
3.5.1. Giới hạn xử lý hạ tầng 34
3.5.2. Thuật toán khử trùng lặp phim (`deduplicateMovies`) . . 34
3.6. Tổ chức phát triển và tiến độ thực hiện 35
3.6.1. Bảng phân công trách nhiệm và minh chứng Git đối soát . . . 35
3.6.2. Tiến độ thực hiện theo tuần (Kế hoạch 9 tuần) 35
3.6.3. Quy trình làm việc nhóm với Git Flow 36

4 KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ 38
4.1. Phương pháp và phân định hai cấu hình Build (Normal vs Mock Evidence) 38
4.2. Kiểm tra tự động và khả năng chạy lại 41
4.3. Bằng chứng về mô hình render và cache 42
4.3.1. Ranh giới module và giới hạn đo bundle 42
4.3.2. Kiểm chứng Data Cache bằng bộ đếm origin 43
4.3.3. Streaming và Watchlist qua HTTP production 43
4.4. Ma trận đáp ứng yêu cầu môn học 44

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 47

TÀI LIỆU THAM KHẢO 48

PHỤ LỤC **50**

Phụ lục 5. Hands-on Lab: Streaming và Data Cache 53

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Thuật ngữ	Định nghĩa và Giải thích chuyên môn
API	*Application Programming Interface* – Giao diện lập trình ứng dụng cho phép các hệ thống giao tiếp với nhau.
CLS	*Cumulative Layout Shift* – Chỉ số đo lường mức độ ổn định bố cục thị giác của trang web (tránh hiện tượng gián đoạn, giật màn hình khi tải ảnh).
CSR	*Client-Side Rendering* – Mô hình kết xuất phía máy khách truyền thống trong React SPA, trình duyệt nhận tệp HTML rỗng và tự nạp JavaScript để sinh cây giao diện DOM.
DNS	*Domain Name System* – Hệ thống phân giải tên miền mạng thành địa chỉ IP máy chủ.
DOM	*Document Object Model* – Mô hình đối tượng tài liệu dạng cây biểu diễn cấu trúc trang web của trình duyệt.
FCP	*First Contentful Paint* – Thời điểm trình duyệt bắt đầu vẽ phần tử nội dung DOM đầu tiên lên màn hình.
ISR	*Incremental Static Regeneration* – Cơ chế tái tạo trang tĩnh định kỳ theo thời gian chỉ định (revalidate) mà không cần biên dịch lại toàn bộ dự án.
LCP	*Largest Contentful Paint* – Chỉ số đo thời gian kết xuất phần tử nội dung trực quan lớn nhất trong khung nhìn người dùng.
PR	*Pull Request* – Yêu cầu đóng góp và hợp nhất nhánh tính năng vào nhánh chính có kiểm duyệt mã nguồn chéo.
REST	*Representational State Transfer* – Kiểu kiến trúc tiêu chuẩn cho các dịch vụ web giao tiếp qua giao thức HTTP.
RPC	*Remote Procedure Call* – Giao thức cho phép gọi trực tiếp một hàm thực thi từ xa trên máy chủ qua mạng, nền tảng của Server Actions.
RSC	*React Server Components* – Component chạy trên máy chủ; mã chỉ dùng ở server không phải tải như JavaScript của Client Component.

Thuật ngữ	Định nghĩa và Giải thích chuyên môn
SEO	*Search Engine Optimization* – Quá trình tối ưu hóa mã nguồn và nội dung để tăng thứ hạng hiển thị trên công cụ tìm kiếm.
SPA	*Single Page Application* – Ứng dụng web đơn trang kết xuất và định tuyến hoàn toàn tại trình duyệt client.
SSG	*Static Site Generation* – Mô hình tạo sẵn toàn bộ tài liệu HTML tĩnh ngay từ thời điểm biên dịch ứng dụng (Build-time rendering).
SSR	*Server-Side Rendering* – Mô hình kết xuất HTML động từ máy chủ cho mỗi lượt truy cập (Request-time rendering).
TMDB	*The Movie Database* – Nguồn cung cấp RESTful API công cộng dữ liệu phim điện ảnh và truyền hình trực tuyến.
TTFB	*Time to First Byte* – Thời gian phản hồi từ lúc client gửi request đến khi nhận được byte dữ liệu đầu tiên từ server.

DANH SÁCH BẢNG

1.1	So sánh đặc trưng kỹ thuật giữa CSR, Pages Router và App Router (Nguồn: Tổng hợp từ [1], [2], [7])	6
1.2	Ma trận đối sánh kỹ thuật giữa Next.js 16 và các giải pháp web hiện đại (Nguồn: Nhóm tác giả tổng hợp)	7
2.1	Đặc tả các yêu cầu chức năng chính của hệ thống MovieHub (Nguồn: Nhóm tác giả đề xuất)	20
2.2	Thời hạn revalidate của các yêu cầu TMDB trong tầng services .	24
2.3	Bảng mã màu và thông số thiết kế Netflix Dark Cinema (Nguồn: Trích xuất từ tài liệu DESIGN.md của dự án)	25
3.1	Nhiệm vụ theo hồ sơ phân công và mốc Git đối chiếu	35
3.2	Các nhóm công việc trong kế hoạch 9 tuần và hiện vật tương ứng	36
4.1	Nguồn minh chứng và phạm vi sử dụng	38
4.2	Các kiểm tra có thể thực thi lại trên mã nguồn hiện tại	41
4.3	Phân loại route từ build Webpack production mock SP07	42
4.4	Bộ đếm mock origin cho mỗi endpoint của một ID phim mới . .	43
4.5	Bảng phủ yêu cầu, nguồn minh chứng và trạng thái	44
PL5.1	Các file cần đọc trong Bài 1	55
PL5.2	Thông tin nhóm thực hành — dành để người học điền	59
PL5.3	Đối chiếu trước/sau sửa — người học điền	60
PL5.4	Marker của một lượt solution — người học điền	60
PL5.5	Counter cùng một ID — người học điền	61
PL5.6	Nội dung sau refresh — người học điền	61
PL5.7	Hai lỗi của bản hướng dẫn trước và cách xử lý	62
PL5.8	Chẩn đoán khi người học gặp trở ngại	62
PL5.9	Đáp án vòng đời Data Cache trong điều kiện mock	64
PL5.10	Streaming mẫu của lượt LB07 — không thay kết quả người học	65

DANH SÁCH HÌNH VẼ

1.1 Sơ đồ luồng kết xuất React Server Components và đóng gói RSC Payload (Nguồn: Xây dựng dựa trên đặc tả kỹ thuật RSC [7]) . . 10
1.2 Sơ đồ phân định ranh giới Server/Client và luồng truyền dữ liệu giữa `MovieCard` và `MovieRow` (Nguồn: Nhóm tác giả thiết kế) 12
2.1 Sơ đồ kiến trúc 3 tầng phân lớp hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế) 22
2.2 Sơ đồ luồng điều hướng trải nghiệm người dùng trong hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế) 26
PL5.1 Đáp án luồng dữ liệu trang phim; các mũi tên mô tả quan hệ gọi/truyền dữ liệu 63

LỜI MỞ ĐẦU

1. Tính cấp thiết của đề tài. Ứng dụng một trang (SPA) kết xuất chủ yếu trên trình duyệt tạo trải nghiệm tương tác linh hoạt, nhưng cần quản lý chi phí JavaScript và cách cung cấp nội dung ban đầu. Các vấn đề cần xem xét gồm:

- **Chi phí JavaScript:** Trình duyệt phải tải, phân tích cú pháp và thực thi mã dành cho các thành phần tương tác; chi phí này phụ thuộc vào từng ứng dụng.
- **Thời gian hiển thị:** Nếu nội dung chỉ được tạo sau khi JavaScript chạy, FCP và LCP có thể bị ảnh hưởng; cần đo trên cùng thiết bị và điều kiện mạng để kết luận.
- **Khả năng đọc nội dung và metadata:** HTML có sẵn từ máy chủ giúp trình thu thập và công cụ xem trước liên kết tiếp cận nội dung ban đầu; mức độ hỗ trợ JavaScript của từng công cụ không giống nhau.
- **Quản lý khóa dịch vụ:** Nếu trình duyệt gọi API bên thứ ba bằng khóa bí mật, khóa đó sẽ hiện diện trong yêu cầu mạng; các yêu cầu cần khóa nên thực hiện trên máy chủ.

Next.js [1] với App Router và React Server Components (RSC) [7] cho phép tạo nội dung trên máy chủ và truyền kết quả xuống trình duyệt. Mã của Server Component không cần đưa vào JavaScript bundle của client, nhưng ứng dụng vẫn cần JavaScript cho Client Components và cơ chế điều hướng. Việc giữ khóa TMDB trong tầng dịch vụ phía server giảm nguy cơ lộ khóa qua mã và yêu cầu mạng của trình duyệt.

Quán triệt tư tưởng chỉ đạo của học phần Các Công nghệ Lập trình Hiện đại: *"Trọng tâm của môn học là làm chủ CÔNG NGHỆ, không phải sa đà vào nghiệp vụ phức tạp; ứng dụng xây dựng chỉ là phương tiện để áp dụng và minh họa"*, nhóm sinh viên chúng em thực hiện đề tài: **"Tìm hiểu công nghệ Next.js qua Ứng dụng tra cứu & khám phá phim ảnh"**. Bằng việc lựa chọn nguồn dữ liệu có sẵn từ The Movie Database (TMDB) API v3 [3], toàn bộ nguồn lực và công sức của nhóm được tập trung tuyệt đối vào việc làm chủ các cơ chế kỹ thuật cốt lõi của Next.js.

2. Mục tiêu nghiên cứu của đồ án. Đồ án bám sát khung chuẩn đánh giá 3 tầng của môn học:

1. **Tầng 1A (Mục tiêu bắt buộc lõi):**
    - Nắm vững cấu trúc thư mục App Router định tuyến theo tệp tin (`app/layout.tsx`, `app/page.tsx`).

- Phân định rõ ràng ranh giới giữa Server Components (chạy mặc định trên máy chủ) và Client Components (khai báo chỉ thị `"use client"`).
- Gọi TMDB tại tầng dịch vụ phía máy chủ và không truyền khóa API xuống Client Components.

2. **Tầng 1B (Mục tiêu nâng cao theo đặc thù ứng dụng):**
   - Làm chủ định tuyến động (Dynamic Routing) `/movie/[id]` xử lý tham số bất đồng bộ dạng Promise (bắt đầu từ Next.js 15 và trở thành quy chuẩn bắt buộc trong Next.js 16).
   - Triển khai Streaming UI với React Suspense và fallback cho các phần dữ liệu phụ; đánh giá thời điểm hiển thị bằng trace khi có dữ liệu đo.
   - Ứng dụng Data Cache có thời hạn và revalidation trong tầng dịch vụ, tách biệt với cache HTML toàn route.
   - Xây dựng các chức năng tương tác dữ liệu bảo mật thông qua Server Actions (`"use server"`) đọc/ghi Cookie an toàn.
   - Dùng `next/image` và khung ảnh có tỉ lệ xác định để hạn chế dịch chuyển bố cục.
   - Tự động hóa SEO thông qua hàm `generateMetadata` trích xuất thông tin động.
3. **Tầng 2 (Mục tiêu chuẩn kỹ nghệ phần mềm):**
   - Quản lý mã nguồn theo mô hình Git Flow phân nhánh chuyên nghiệp, kiểm duyệt mã thông qua Pull Request.
   - Đảm bảo tính nhất quán và an toàn mã nguồn với TypeScript chế độ nghiêm ngặt (Strict Mode).
   - Xử lý lỗi mạng và xác định giới hạn xử lý hạ tầng trong Node.js.
   - Tuân thủ nghiêm ngặt hệ thống thiết kế chuẩn điện ảnh Netflix Dark Cinema (`DESIGN.md`).

3. Đối tượng và phạm vi nghiên cứu.

- **Đối tượng nghiên cứu:** Nền tảng Next.js phiên bản 16.4.5 (App Router), thư viện React 19.2.8, ngôn ngữ TypeScript, framework CSS tiện ích Tailwind CSS v4, và giao diện lập trình ứng dụng The Movie Database (TMDB) REST API v3.
- **Phạm vi ứng dụng:** Xây dựng ứng dụng web **MovieHub** phục vụ việc tra cứu, tìm kiếm, lọc đa tiêu chí, xem chi tiết và quản lý danh sách theo dõi phim ảnh.

Ứng dụng tập trung khai thác và trình diễn các kỹ thuật Next.js; không đi sâu vào việc phát triển hạ tầng backend cơ sở dữ liệu riêng mà tận dụng nguồn dữ liệu phong phú từ TMDB.

- **Thời gian thực hiện:** Dự án được tiến hành trong thời gian 7 tuần theo kế hoạch đào tạo của học phần.

4. Phương pháp nghiên cứu.

- **Phương pháp nghiên cứu lý thuyết:** Nghiên cứu tài liệu chính thống (Official Documentation) từ Vercel, React Core Team và W3C; phân tích các thông số kỹ thuật (RFC) của React Server Components và cơ chế HTTP Streaming.
- **Phương pháp thực nghiệm phát triển:** Áp dụng quy trình kỹ nghệ phần mềm chuyên nghiệp, xây dựng mã nguồn từ cơ bản đến nâng cao theo từng sprint tuần, kiểm thử đối chứng trên môi trường thực tế.
- **Phương pháp kiểm chứng:** Đối chiếu mã với yêu cầu; chạy lint, type-check, test và production build. Các phép đo Lighthouse, bundle và TTFB được mô tả như công việc tiếp theo vì chưa có tệp kết quả gốc.

5. Bố cục của báo cáo đồ án. Nội dung báo cáo được tổ chức thành 4 chương chính như sau:

- **Chương 1: Cơ sở lý thuyết và công nghệ sử dụng** – Trình bày tổng quan về Next.js, kiến trúc App Router, React Server Components, cơ chế Caching, Streaming, Server Actions và các công nghệ phụ trợ.
- **Chương 2: Phân tích và thiết kế hệ thống** – Đặc tả các yêu cầu chức năng và phi chức năng, thiết kế kiến trúc phân lớp, cấu trúc thư mục, luồng dữ liệu, phạm vi xử lý lỗi mạng và hệ thống thiết kế UI/UX.
- **Chương 3: Hiện thực hệ thống** – Chi tiết quá trình cài đặt, hiện thực các tính năng Tầng 1A, Tầng 1B, các module giao diện nổi bật, cơ chế chịu lỗi mạng, cùng quy trình phân công và tiến độ thực hiện.
- **Chương 4: Kiểm thử và đánh giá kết quả** – Trình bày bằng chứng kiểm chứng hiện có, giới hạn của phép kiểm thử và ma trận đối sánh yêu cầu môn học.
- **Kết luận và hướng phát triển** – Tổng kết các kết quả đạt được, chỉ ra những hạn chế và đề xuất hướng mở rộng trong tương lai.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

1.1. Tổng quan về Next.js và lịch sử phát triển

1.1.1. Bối cảnh ra đời và hạn chế của React Single Page Application (SPA)

Kể từ khi được Facebook giới thiệu vào năm 2013, thư viện React đã tạo nên cuộc cách mạng trong thiết kế giao diện nhờ mô hình Virtual DOM và tư duy phát triển dựa trên Component. Tuy nhiên, mô hình ứng dụng đơn trang (SPA) dựa trên cơ chế Client-Side Rendering (CSR) đã bộc lộ những nhược điểm lớn khi mở rộng quy mô:

1. **Hiện tượng trang trắng ban đầu (Blank Page):** Trình duyệt nhận về tệp tin HTML gần như rỗng chỉ chứa một phần tử `<div id="root"></div>`. Người dùng phải đợi trình duyệt tải xong toàn bộ gói mã nguồn JavaScript, sau đó chạy mã để dựng cây DOM thực tế.
2. **Tải trọng mạng và chi phí xử lý của thiết bị di động:** Các gói thư viện phụ thuộc (npm packages) đều bị đóng gói dồn vào bundle của client. Trên các thiết bị di động cấu hình yếu, việc phân tích cú pháp (parse) và biên dịch (compile) tệp JavaScript nặng hàng megabyte gây đơ giật giao diện và kéo dài chỉ số Time to Interactive (TTI).
3. **Thách thức về SEO và Social Sharing:** Các hệ thống lập chỉ mục tìm kiếm và robot thu thập dữ liệu mạng xã hội (Open Graph) thường không thực thi hoàn chỉnh các luồng JavaScript bất đồng bộ, khiến website bị xếp hạng thấp hoặc không hiển thị được ảnh đại diện xem trước.

1.1.2. Bước chuyển đổi với Pages Router (Next.js 1 – 12)

Nhằm khắc phục các điểm yếu của React SPA, Vercel đã giới thiệu nền tảng Next.js vào năm 2016 [1]. Kiến trúc Pages Router tổ chức các trang web tương ứng với các tệp tin trong thư mục `pages/`, đồng thời cung cấp hai cơ chế tiền kết xuất mạnh mẽ:

- **Server-Side Rendering (SSR) với `getServerSideProps`:** Máy chủ thực thi lấy dữ liệu và kết xuất mã HTML hoàn chỉnh cho từng request của người dùng trước khi gửi về client.
- **Static Site Generation (SSG) với `getStaticProps`:** Máy chủ tạo sẵn tài liệu HTML ngay tại thời điểm biên dịch ứng dụng (Build-time), mang lại tốc độ phản hồi cực nhanh thông qua mạng phân phối nội dung (CDN).

Tuy nhiên, Pages Router vẫn tồn tại hạn chế về mặt kiến trúc gọi là **Chi phí Hydration toàn bộ (Hydration Overhead)**. Sau khi máy chủ gửi tài liệu HTML về client, trình duyệt vẫn bắt buộc phải tải về toàn bộ mã nguồn JavaScript của toàn bộ trang để tiến hành quá trình "Hydrate" – gắn các trình lắng nghe sự kiện (Event Listeners) vào các nút DOM tĩnh. Điều này đồng nghĩa với việc mã nguồn của các component chỉ hiển thị nội dung tĩnh (như Header, Footer, Sidebar) vẫn bị gửi xuống client một cách dư thừa.

1.1.3. Kỷ nguyên mới với App Router và Server-Driven UI (Next.js 13 – 16)

Nhận thức được giới hạn của kiến trúc cũ, đội ngũ kỹ sư Vercel phối hợp chặt chẽ cùng React Core Team đã phát triển kiến trúc **App Router**, được chính thức đưa vào sản xuất từ Next.js 13 và hoàn thiện vững chắc trong Next.js 16 [1].

App Router được xây dựng xung quanh khái niệm **React Server Components (RSC)** [7]. Trọng tâm kết xuất được chuyển dịch triệt để về phía máy chủ (Server-Driven UI). Trong mô hình này, các component mặc định chỉ chạy trên máy chủ, cho phép truy cập trực tiếp vào hệ thống tệp, cơ sở dữ liệu và biến môi trường bảo mật mà không gửi một dòng mã lệnh JavaScript nào của component đó xuống trình duyệt.

Bảng 1.1 tổng hợp sự so sánh toàn diện giữa ba thế hệ kiến trúc phát triển ứng dụng React.

Bảng 1.1. So sánh đặc trưng kỹ thuật giữa CSR, Pages Router và App Router (Nguồn: Tổng hợp từ [1], [2], [7])

Tiêu chí so sánh	React CSR (SPA)	Next.js Pages Router	Next.js App Router
Vị trí kết xuất chính	Trình duyệt Client	Server & Client Hydrate	Server (mặc định) + Client
Tải trọng JS Client	Rất lớn (toàn bộ app)	Lớn (toàn trang nạp)	Tối ưu (Zero-bundle RSC)
Khả năng SEO	Rất kém (HTML rỗng)	Tốt (SSR/SSG tĩnh)	Tối ưu xuất sắc
Cơ chế Caching & Mặc định fetch	Cache trình duyệt thủ công	HTTP Cache đơn giản	Next.js 16 mặc định `no-store` (khác Next.js 14 `force-cache`); chỉ lưu đệm khi chỉ định tường minh Data Cache (`revalidate`) hoặc Cache Components (“`use cache`”)
Xác định Route Tĩnh / Động	Toàn bộ tĩnh (SPA)	Thủ công qua `getStaticProps` (SSG) vs `getServerSideProps` (SSR)	Tự động suy luận: Tĩnh (○) nếu không dùng Dynamic API và các fetch đều cache; Động (*ƒ*) nếu gọi `cookies()`, `headers()` hoặc có fetch không cache
Hỗ trợ Streaming	Không hỗ trợ	Không có	React Suspense Streaming
Xử lý Mutation	Gọi REST API Route	Tạo thư mục `pages/api`	Server Actions trực tiếp
Bảo mật API Key	Dễ lộ trong mã client	Phải qua API trung gian	Giữ hoàn toàn trên Server

1.1.4. So sánh Next.js App Router với các giải pháp hiện đại khác

Để có cái nhìn khách quan và toàn diện về bức tranh công nghệ web hiện đại, Bảng 1.2 đối sánh Next.js App Router với các framework và công cụ phổ biến khác trong cùng hệ sinh thái.

Bảng 1.2. Ma trận đối sánh kỹ thuật giữa Next.js 16 và các giải pháp web hiện đại (Nguồn: Nhóm tác giả tổng hợp)

Tiêu chí đối sánh	Next.js 16 (App Router)	React 19 + Vite	Nuxt 4 (Vue.js 3)	SvelteKit (Svelte 5)	Astro (Islands)
Khả năng SSR/SSG/ISR	Toàn diện: RSC, SSR động, SSG tĩnh, và ISR qua Data Cache `revalidate`	Thuần CSR mặc định; cần plugin ngoài (Vike) để SSR; không có ISR gốc	Hỗ trợ SSR, SSG và ISR/SWR linh hoạt qua Nitro Engine (`routeRules`)	Hỗ trợ SSR, SSG (`prerender`), và ISR qua Edge Adapters	Mặc định SSG tĩnh; hỗ trợ on-demand SSR qua adapter; ISR phụ thuộc CDN
Tải trọng Client JS	Tối ưu nhờ Zero-Bundle RSC; chỉ hydrate các Client Components	Lớn nhất; 100% mã ứng dụng và thư viện phải nạp về client để hydrate	Trung bình; nạp runtime Vue, router và code component để full hydrate	Siêu nhỏ (<20 KB); biên dịch trực tiếp ra DOM code, 0 Virtual DOM runtime	Cực nhỏ (Zero JS mặc định); chỉ hydrate các ốc đảo tương tác (Islands)
Trải nghiệm lập trình (DX)	Turbopack biên dịch nhanh; tư duy phân tách ranh giới mạng phức tạp	DX xuất sắc; HMR tức thì qua Vite ESM, mô hình React thuần dễ debug	Rất tốt; tự động nạp composables, tích hợp sâu hệ sinh thái Vue	Xuất sắc; cú pháp Svelte thanh thoát, Runes ($state) tinh gọn, ít boilerplate	Rất tốt cho site nội dung; cú pháp template trực quan, hỗ trợ đa framework
Quản lý trạng thái Server	Tích hợp Server Actions (`"use server"`) và RSC; Route Handlers	Dùng thư viện ngoài: TanStack Query, SWR, Axios kết hợp Redux/Zustand	Tích hợp sẵn `useFetch`, `useAsyncData` và server routes Nitro	Form Actions nguyên bản (`+page.server.ts`), progressive enhancement	Sử dụng top-level await trên server và Astro Actions; state qua nanostores
Đường cong học tập	Dốc đứng; đòi hỏi hiểu sâu về ranh giới môi trường và Caching đa tầng	Thoải mái; mô hình React truyền thống quen thuộc	Trung bình; quy ước rõ ràng, dễ tiếp cận hơn App Router	Thấp - trung bình; cú pháp HTML/JS tự nhiên, dễ nắm bắt hơn RSC	Thấp nhất; thân thiện với lập trình viên web căn bản

1.1.5. Phản biện kỹ thuật: Khi nào KHÔNG nên sử dụng Next.js?

Dựa trên trải nghiệm thực tế trong quá trình phát triển dự án MovieHub, nhóm nhận thấy Next.js không phải là giải pháp vạn năng cho mọi bài toán công nghệ. Dưới đây

là các kịch bản thực tế mà việc sử dụng Next.js mang lại nhiều bất lợi hơn là ưu điểm:

1. **Ứng dụng nội bộ hoặc Bảng điều khiển quản trị (Internal Dashboard / CMS sau đăng nhập):** Các hệ thống này nằm hoàn toàn sau bức tường xác thực, không có nhu cầu SEO hay Open Graph. Việc áp dụng SSR làm tăng chi phí máy chủ và phát sinh nguy cơ Hydration Mismatch; kiến trúc React SPA thuần với Vite và TanStack Query là phương án vượt trội về tốc độ phát triển và tiết kiệm chi phí vận hành.
2. **Website nội dung tĩnh đơn giản (Blog cá nhân, Landing Page, Tài liệu hướng dẫn):** Phục vụ nội dung tĩnh qua máy chủ Node.js là lãng phí tài nguyên. Công cụ như Astro xuất 100% HTML tĩnh với Zero JavaScript mặc định, triển khai miễn phí trên CDN toàn cầu.
3. **Ứng dụng thời gian thực liên tục (Real-time Collaborative Apps / WebSockets):** Mô hình Serverless/Request-Response ngắn hạn của Next.js không tối ưu cho các kết nối TCP/WebSocket kéo dài (như Figma, Google Docs).

Các thách thức kỹ thuật cố hữu của nền tảng Next.js:.

- **Độ trễ khởi động lạnh (Cold Starts) trên môi trường Serverless:** Các tuyến đường kết xuất động (Dynamic SSR) và Route Handler khi triển khai trên hạ tầng Serverless (AWS Lambda, Vercel Functions) phải chịu độ trễ khởi động tiến trình Node.js (từ 200ms đến hơn 1.500ms) khi có yêu cầu đột biến sau thời gian nhàn rỗi.
- **Rủi ro phụ thuộc nền tảng (Vendor Lock-in) và chi phí tự vận hành (Self-Hosting):** Các tính năng cao cấp như Incremental Cache, Image Optimization, và On-Demand ISR được thiết kế tối ưu hóa riêng cho hệ sinh thái thương mại của Vercel. Khi tự triển khai trên máy chủ độc lập (Docker/Kubernetes), đội ngũ kỹ thuật phải tự cấu hình phức tạp với standalone output, Redis Cache Handler (`@neshca/cache-handler`), làm tăng đáng kể chi phí bảo trì DevOps.
- **Độ phức tạp và sự chuyển dịch của mô hình Caching:** Từ phiên bản Next.js 15 và 16, Vercel đã xóa bỏ cơ chế tự động lưu đệm ngầm (`force-cache` mặc định thời Next.js 14) vốn dễ gây hiện tượng dữ liệu cũ ngoài ý muốn (stale data). Thay vào đó, Next.js chuyển sang cơ chế tường minh: mặc định hàm `fetch` là `no-store`. Ở Next.js 16, Vercel chuẩn hóa kiến trúc **Cache Components** dưới cấu hình gốc `cacheComponents: true` (hợp

nhất các cờ tiền thân như `ppr`, `useCache`, và `dynamicIO`). Khi cấu hình này tắt (`cacheComponents: false`) như thiết lập mặc định trong dự án MovieHub, ứng dụng tuân thủ mô hình Caching tiêu chuẩn (Previous Caching Model), trong đó bộ đệm dữ liệu được điều khiển tường minh ở cấp độ yêu cầu mạng (Request-level Data Cache) thông qua tùy chọn `next: { revalidate: N }`. Lập trình viên phải phân định rạch ròi giữa Data Cache và các Dynamic Functions (như `cookies()`) để tránh vô tình chuyển đổi tuyến tĩnh thành tuyến động.

1.2. Kiến trúc React Server Components (RSC) và Client Components

1.2.1. Bản chất và nguyên lý hoạt động của RSC

Khác với component React truyền thống (chạy trên cả server và client trong quá trình SSR), **React Server Component (RSC)** là một loại component đặc biệt chỉ thực thi duy nhất trên môi trường máy chủ [7].

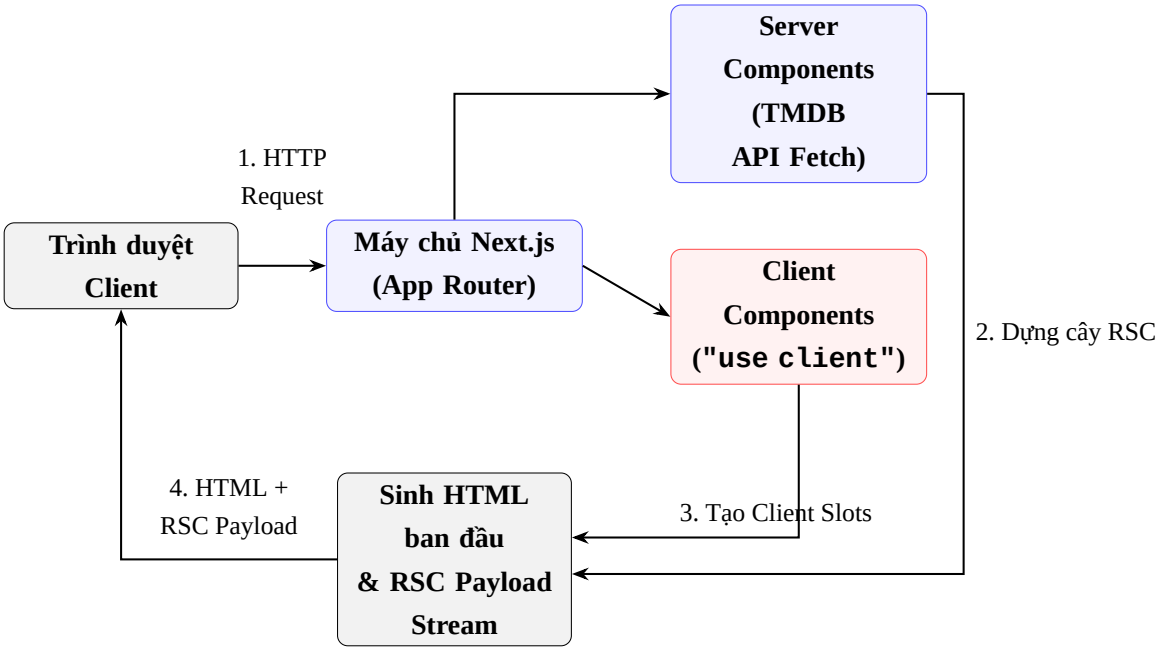
Khi một yêu cầu được gửi tới máy chủ Next.js:

1. Máy chủ thực thi hàm component (hỗ trợ hàm bất đồng bộ `async/await` trực tiếp ở cấp cao nhất).
2. Máy chủ gọi dữ liệu từ cơ sở dữ liệu hoặc dịch vụ bên ngoài (TMDB API) mà không cần dùng hook `useEffect`.
3. Thay vì sinh ra mã JavaScript để gửi về trình duyệt, máy chủ tuần tự hóa (serialize) cây component thành một luồng dữ liệu nhị phân/văn bản có cấu trúc gọi là **RSC Payload**.
4. RSC Payload chứa cấu trúc của cây giao diện, các thuộc tính props được truyền và vị trí đánh dấu (placeholders) nơi các Client Component cần được nạp và hydrate.

Các module chỉ được Server Components nhập sẽ không tham gia vào module graph của Client Components. Tuy nhiên, thư viện được một Client Component nhập trực tiếp vẫn có thể vào client bundle; kích thước thực tế cần kiểm tra bằng analyzer.

1.2.2. Định dạng dữ liệu RSC Payload và cây render hỗn hợp

Hình 1.1 mô tả chi tiết luồng xử lý và sự tương tác giữa Server Components, Client Components và định dạng dữ liệu RSC Payload.



Hình 1.1. Sơ đồ luồng kết xuất React Server Components và đóng gói RSC Payload (Nguồn: Xây dựng dựa trên đặc tả kỹ thuật RSC [7])

1.2.3. Chỉ thị `"use client"` và quy tắc Composition Pattern

Trong Next.js App Router, mọi tệp tin component mặc định là Server Component. Để chuyển đổi một component sang Client Component, lập trình viên phải đặt chỉ thị `"use client"` ở dòng đầu tiên của tệp tin. Chỉ thị này xác định một **Ranh giới mạng (Network Boundary)**, báo hiệu cho trình biên dịch rằng toàn bộ component này cùng các module được nó import sẽ được đóng gói gửi về client để thực hiện hydration.

Các trường hợp bắt buộc phải sử dụng Client Component bao gồm:

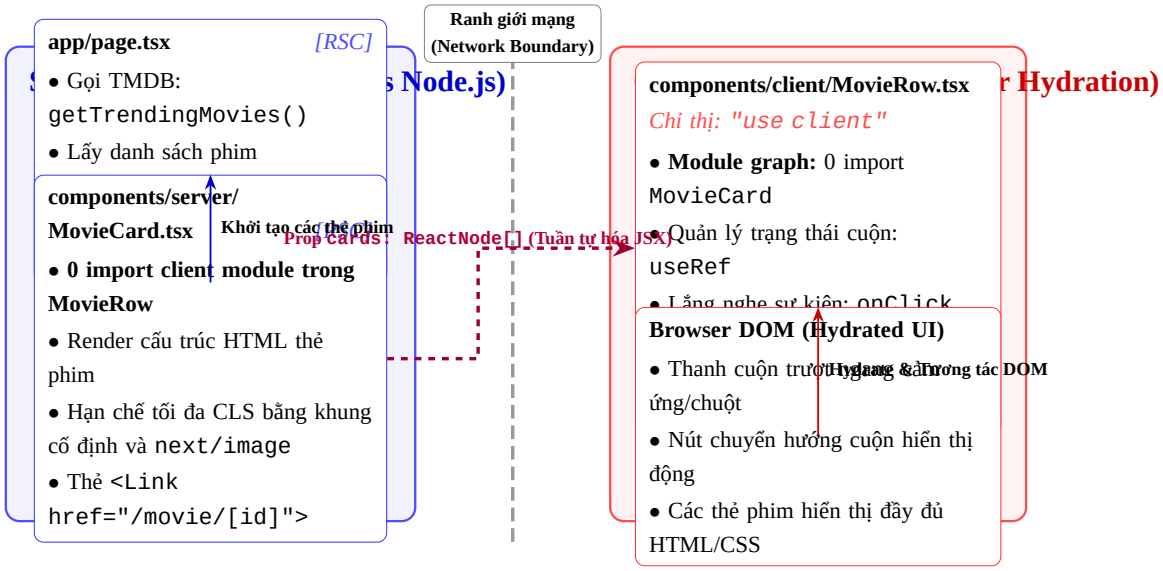
- Cần lắng nghe sự kiện tương tác của người dùng: `onClick`, `onChange`, `onSubmit`.
- Cần quản lý trạng thái động hoặc vòng đời: `useState`, `useReducer`, `useEffect`.
- Cần truy cập các API riêng của trình duyệt: `localStorage`, `sessionStorage`, `window`, `document`.
- Cần sử dụng các React Custom Hooks quản lý giao diện tương tác.

Để duy trì hiệu năng tối ưu, đồ án tuân thủ nghiêm ngặt mô hình **Composition Pattern**: Đẩy các Client Component xuống các nút lá nhỏ nhất của cây giao diện (ví dụ: nút bấm mở modal `TrailerModal`, thanh cuộn `MovieRow`), giữ cho các

component cha (như `app/page.tsx`, `app/layout.tsx`) là Server Components để thực hiện lấy dữ liệu tập trung.

Ranh giới Server/Client và cơ chế Zero-Bundle Slider trong MovieHub:. Hình 1.2 trực quan hóa cơ chế phân định ranh giới mạng (Network Boundary) giữa Server Component và Client Component tại Trang chủ của MovieHub:

1. **Phía máy chủ (Server Environment):** Tuyến đường `app/page.tsx` là một React Server Component (RSC), thực thi độc quyền trên môi trường Node.js. Nó gọi hàm dịch vụ `getTrendingMovies()` để lấy danh sách phim, sau đó khởi tạo trực tiếp danh sách các thẻ `<MovieCard movie={m} />` trên server. Component `MovieCard.tsx` cũng là một Server Component, không chứa chỉ thị `"use client"`. Nhờ đó, toàn bộ mã nguồn xử lý layout, các thẻ cấu trúc, và thư viện `next/image` được kết xuất hoàn toàn thành HTML tĩnh mà **không đưa một dòng mã JavaScript nào của `MovieCard` vào client bundle** (đạt chuẩn 0 KB JavaScript client bundle).
2. **Vượt qua ranh giới mạng (Across Network Boundary):** Thay vì nhập khẩu trực tiếp tệp `MovieCard` vào phía client, `app/page.tsx` truyền danh sách các phần tử React đã render sẵn qua thuộc tính `cards={movies.map(...)}` (dưới dạng mảng `ReactNode[]`). Dữ liệu này được Next.js tuần tự hóa an toàn trong luồng RSC Payload.
3. **Phía trình duyệt (Client Environment):** Component `components/client/MovieRow.tsx` mang chỉ thị `"use client"` đóng vai trò là Client Boundary chuyên biệt. Nó chỉ chịu trách nhiệm quản lý tương tác giao diện người dùng: sử dụng hook `useRef` để gắn kết container cuộn ngang, xử lý sự kiện nhấp chuột (`onClick`) trượt trái/phải, và đặt các phần tử `props.cards` vào trong thanh trượt. Bộ kiểm thử kiến trúc tự động (`tests/server-client-boundary.test.mjs`) giám định và khẳng định số lượng import của `MovieCard` trong mã nguồn client của `MovieRow` bằng đúng **0**, bảo đảm tuyệt đối tính toàn vẹn của ranh giới mạng.



Hình 1.2. Sơ đồ phân định ranh giới Server/Client và luồng truyền dữ liệu giữa `MovieCard` và `MovieRow` (Nguồn: Nhóm tác giả thiết kế)

1.3. Cơ chế Caching và Data Fetching đa tầng trong Next.js 16

Dự án MovieHub được xây dựng trên nền tảng **Next.js 16.4.5** và **React 19.2.8**. Một trong những thay đổi quan trọng nhất của Next.js kể từ phiên bản 15 và 16 là sự chuyển dịch mô hình bộ nhớ đệm: Hàm `fetch` mặc định không còn tự động lưu đệm vĩnh viễn (`force-cache`) như thời Next.js 14, mà chuyển sang mặc định `no-store` trừ khi lập trình viên chủ động khai báo chiến lược lưu trữ.

1.3.1. Tính hợp lệ của API `fetch` và tùy chọn `revalidate`

Một số quan điểm cho rằng cú pháp `fetch(url, { next: { revalidate: seconds } })` là cơ chế cũ của Next.js 13/14 và bị thay thế hoàn toàn trong Next.js 16. Tuy nhiên, theo tài liệu đặc tả chính thức của Vercel (https://nextjs.org/docs/app/api-reference/functions/fetch), đây vẫn là **API chuẩn mực, ổn định (Stable API) và hoàn toàn hợp lệ** trong Next.js 16 để điều khiển **Data Cache ở cấp độ yêu cầu mạng (Request-level Caching)**:

$$\text{fetch}(\text{url}, \{\, \text{next} : \{\, \text{revalidate} : 3600 \,\} \,\}) \tag{1.1}$$

Nguyên lý hoạt động theo cơ chế **Stale-While-Revalidate**:

- Trong khoảng thời gian revalidate (ví dụ 3600 giây), yêu cầu có cùng khóa cache có thể dùng lại dữ liệu đã lưu; báo cáo chưa đo độ trễ cache hit.

- Sau khi hết hạn TTL, khi có lượt truy cập mới, máy chủ phục vụ bản dữ liệu cũ trước (stale), đồng thời kích hoạt một tiến trình nền bất đồng bộ (Background Revalidation) để gọi API TMDB lấy dữ liệu mới và cập nhật bản ghi cache.
- Lượt truy cập tiếp theo sẽ nhận được dữ liệu hoàn toàn mới mà không gây nghẽn luồng xử lý chính.

1.3.2. Cơ chế Cache Components ("`use cache`" và `cacheLife`) trong Next.js 16

Next.js 16 chính thức giới thiệu kiến trúc bộ nhớ đệm cấp độ component mang tên **Cache Components** thông qua cờ cấu hình gốc `cacheComponents: true` trong tệp `next.config.ts`. Đây là quy chuẩn thống nhất thay thế và hợp nhất các cờ tiền thân riêng lẻ trước đây (`ppr`, `useCache`, và `dynamicIO`):

- **Trạng thái trong dự án MovieHub:** Trong tệp cấu hình `next.config.ts`, MovieHub thiết lập `cacheComponents: false`. Do đó, ứng dụng hoạt động hoàn toàn theo mô hình Caching tiêu chuẩn của Next.js (Previous Caching Model). Toàn bộ dữ liệu phim TMDB được lưu đệm tường minh ở cấp độ lời gọi mạng thông qua `fetch(url, { next: { revalidate: N } })`, không kích hoạt chỉ thị `"use cache"` hay hàm `cacheLife()`.
- **Cơ chế hoạt động khi được kích hoạt (`cacheComponents: true`):** Khi bật cờ gốc, lập trình viên sử dụng chỉ thị `"use cache"` ở đầu async function hoặc component, kết hợp hàm định nghĩa vòng đời `cacheLife()` với các hồ sơ cấu hình chuẩn:
  - `cacheLife("seconds")`: `revalidate: 1s, expire: 60s`.
  - `cacheLife("minutes")`: `revalidate: 60s, expire: 1h`.
  - `cacheLife("hours")`: `revalidate: 1h, expire: 24h`.
  - `cacheLife("days")`: `revalidate: 24h, expire: 7d`.
  - `cacheLife("default")`: sử dụng profile mặc định do cấu hình Next.js xác định.

  Ưu điểm của Cache Components là có thể lưu đệm cả kết quả tính toán phức tạp hoặc truy vấn cơ sở dữ liệu trực tiếp ở cấp hàm, không phụ thuộc vào lệnh `fetch`.

1.3.3. Tác động của việc đọc `cookies()` đến quá trình Render và Caching

Trong kiến trúc Next.js App Router, việc truy cập các Dynamic Functions (như `cookies()`, `headers()`) sẽ tự động chuyển tuyến đường đó sang chế độ **Dynamic Rendering (kết xuất động theo từng yêu cầu)**:

- **Các tuyến đường thực tế đọc cookie trong MovieHub:**
  1. Tuyến `/watchlist` (`app/watchlist/page.tsx`): đọc cookie qua `getWatchlistIds()` để lấy danh sách phim của người dùng.
  2. Tuyến `/movie/[id]` (`app/movie/[id]/page.tsx`): gọi `isMovieSaved(id)` để xác định trạng thái ban đầu của nút lưu phim.
  3. Server Action trong `app/actions/watchlist.ts`: dùng `cookies()` để thêm/xóa phim.
- **Header và MovieCard:** Hai component này không đọc cookie. `Header` dùng `scrollY` và `usePathname`; `MovieCard` nhận phim qua props, không đọc trạng thái Watchlist.

1.3.4. Mối quan hệ tương hỗ giữa Streaming Suspense và Data Caching

Kỹ thuật truyền luồng (Streaming với React Suspense) và Caching dữ liệu (ISR) **không hề loại trừ nhau**, mà hỗ trợ lẫn nhau theo hai giai đoạn rõ rệt:

1. **Khi dữ liệu fetch đã có trong Data Cache:** Các lời gọi TMDB tương ứng có thể dùng lại kết quả; điều này không đồng nghĩa toàn bộ HTML của một route động đã được cache.
2. **Khi dữ liệu chưa có trong cache:** Trang `/movie/[id]` có thể phát phần chính cùng fallback Suspense trước khi `CastList` và `SimilarMovies` hoàn tất. Thời điểm từng chunk phụ thuộc dữ liệu và môi trường chạy, cần Network trace để xác nhận định lượng.

1.3.5. Các cơ chế nền tảng bổ trợ: Hydration, Vòng đời Hook và URL State

Để đảm bảo trải nghiệm người dùng hoàn chỉnh, dự án tích hợp chặt chẽ các cơ chế nền tảng bắt buộc của lập trình web hiện đại:

- **Cơ chế Client Hydration và phòng chống Hydration Mismatch:** Sau khi trình duyệt nhận được HTML render sẵn từ server, React gắn kết các bộ lắng nghe

sự kiện (event listeners) vào DOM. Để tránh lỗi bất đồng bộ giữa cây DOM server và client (Hydration Mismatch), các trạng thái phụ thuộc vào trình duyệt (như trạng thái cuộn chuột `window.scrollY` trong `Header`) chỉ được kích hoạt bên trong hook `useEffect` sau khi component đã mount thành công trên client.

- **Quản lý trạng thái thông qua URL (`searchParams`):** Tại các trang `/browse` và `/search`, toàn bộ trạng thái bộ lọc (thể loại, năm, sắp xếp, trang hiện tại) được lưu trữ trực tiếp trên URL Query Parameters. Cách tiếp cận này giúp người dùng có thể chia sẻ liên kết kết quả lọc, lưu bookmark trình duyệt, và sử dụng các nút Back/Forward mà không bị mất trạng thái. Thư viện hook `useTransition` kết hợp `useRouter.replace()` giúp cập nhật URL mượt mà mà không gây đơ cứng giao diện.
- **Điều hướng với `next/link`:** Các liên kết nội bộ như thẻ phim dùng `<Link>` của Next.js để điều hướng phía client. Next.js có thể prefetch tuyến đường đích tùy loại route và cấu hình; thời gian chuyển trang thực tế còn phụ thuộc dữ liệu và mạng.

1.4. Kỹ thuật Streaming UI và React Suspense

1.4.1. Phạm vi chặn và streaming trong MovieHub

Trang chủ dùng `Promise.all` cho bốn danh mục và đợi toàn bộ tác vụ thành công trước khi trả nội dung trang; lỗi tải dữ liệu được truyền tới error boundary. Đây không phải streaming từng hàng phim.

Trang `/movie/[id]` đợi thông tin phim, trạng thái cookie Watchlist và trailer trước khi dựng phần chính. Chỉ hai nhánh `CastList` và `SimilarMovies` nằm trong Suspense độc lập: fallback được gửi trước nội dung thật khi nhánh tương ứng chậm. Một `Promise.all` chỉ chặn component đang await; loading boundary phía trên vẫn có thể gửi fallback, nên không thể suy ra màn hình trắng hay TTFB của toàn response từ độ trễ API.

Minh chứng `labs/streaming-cache/evidence/rebuilt-full/streaming-mock.json` ghi chunk và thời điểm nhận HTML của lượt production mock hiện tại. Mock chủ động làm chậm credits 2,5 giây và similar 4 giây. Kiểm tra yêu cầu phần chính và fallback đến trước kết quả ít nhất 1000ms. Không dùng các thời điểm này làm FCP, Core Web Vitals hoặc thời điểm trình duyệt đã tương tác được; không có phép đo đối chứng SSR để kết luận mức tăng tốc.

```
Module import: app/page -> MovieCard (server)
               app/page -> MovieRow (client boundary)
               MovieRow -> React hooks; no MovieCard import
Render:        app/page -> cards={<MovieCard ... />} -> MovieRow
```

Props qua RSC phải truyền được qua ranh giới server/client; nội dung React được server dựng rồi truyền vào slider không đưa module MovieCard vào client bundle chỉ vì nằm dưới slider trong cây hiển thị. CastSlider hiện nhận dữ liệu diễn viên và dựng markup phía client; không suy rộng kết luận của MovieRow cho mọi slider.

1.5. Cơ chế Server Actions và xử lý Mutation dữ liệu

1.5.1. Kiến trúc Remote Procedure Call (RPC)

Để thực hiện thao tác thay đổi dữ liệu từ máy khách, một cách triển khai là tạo API endpoint riêng:

1. Tạo một endpoint API Route riêng biệt (ví dụ: `/api/watchlist`).
2. Viết mã kiểm tra phương thức HTTP (POST/DELETE), đọc tham số từ body.
3. Phía client sử dụng lệnh `fetch('/api/watchlist')` với các bước cấu hình headers phức tạp.

Với **Server Actions**, Next.js giới thiệu mô hình gọi thủ tục từ xa (RPC) tích hợp sâu vào cây component. Lập trình viên chỉ cần khai báo một hàm bất đồng bộ với chỉ thị `"use server"`:

```
// app/actions/watchlist.ts (rut gon)
"use server";
export async function toggleWatchlist(movieId: number) {
  if (!Number.isInteger(movieId) || movieId <= 0) {
    throw new Error("movieId khong hop le");
  }
  const cookieStore = await cookies();
  const currentIds = await getWatchlistIds();
  // Them/bo ID, ghi cookie va revalidatePath("/watchlist")
}
```

Next.js cung cấp lời gọi POST để client kích hoạt Action; hàm vẫn phải tự xác thực dữ liệu đầu vào và kiểm soát quyền truy cập nếu có tài khoản người dùng.

1.5.2. Bảo mật dữ liệu, kiểm thực đầu vào và giới hạn Cookie HttpOnly

Server Actions thực thi hoàn toàn trong môi trường máy chủ bảo mật, mang lại các lợi thế kỹ thuật vượt trội nhưng cũng đòi hỏi các biện pháp bảo vệ đi kèm:

- **Cookie HttpOnly:** `httpOnly: true` chặn truy cập qua `document.cookie`; `secure` được bật trong production để chỉ gửi cookie qua HTTPS. Hai cờ này không tự loại bỏ lỗ hổng XSS hoặc mọi rủi ro bảo mật.
- **Kiểm thực đầu vào:** `toggleWatchlist` kiểm tra `movieId` là số nguyên dương. Tìm kiếm gợi ý qua GET Route Handler xác thực và giới hạn chuỗi truy vấn riêng; nó không phải Server Action.
- **Giới hạn Cookie:** Watchlist giới hạn 200 ID và kiểm tra số byte của chuỗi JSON trước khi ghi. Giới hạn thực tế mỗi cookie tùy trình duyệt; đây là danh sách cục bộ trên một thiết bị, không thay thế cơ sở dữ liệu.
- **Làm mới sau thay đổi:** Action gọi `revalidatePath` cho trang Watchlist và trang chi tiết liên quan. Giao diện cần tải lại dữ liệu server để phản ánh cookie mới.

1.6. Hệ sinh thái công nghệ bổ trợ

1.6.1. Thư viện React 19

Dự án được xây dựng trên nền tảng phiên bản mới nhất **React 19.2.8**. Phiên bản này cung cấp các cải tiến vượt trội về cơ chế quản lý trạng thái tương tác:

- Server Action có thể kết hợp `useActionState` trong ứng dụng khác; MovieHub dùng trạng thái cục bộ tại nút Watchlist.
- Hook `useTransition`: Cho phép đánh dấu các tác vụ cập nhật giao diện là tác vụ không khẩn cấp (non-urgent), giữ cho giao diện luôn phản hồi mượt mà với các thao tác của người dùng.

1.6.2. Framework giao diện Tailwind CSS v4

Dự án áp dụng thế hệ mới nhất **Tailwind CSS v4** [5]. Đây là bước tái kiến trúc toàn diện của framework CSS tiện ích này:

- **Loại bỏ tệp cấu hình truyền thống:** Không còn yêu cầu tệp `tailwind.config.js` phức tạp; toàn bộ các biến giao diện, màu sắc chủ đạo, font chữ được định nghĩa trực tiếp trong tệp CSS nguồn thông qua chỉ thị `@theme`.

- **Engine biên dịch Oxide:** Tailwind CSS v4 cung cấp bộ máy xử lý mới; báo cáo không có benchmark tốc độ giữa v3 và v4 trên repo này.
- **Tương thích tự nhiên với biến CSS (CSS Custom Properties):** Dễ dàng xây dựng hệ thống thiết kế đồng nhất theo tiêu chuẩn điện ảnh Netflix Dark Cinema.

1.6.3. Ngôn ngữ lập trình TypeScript

Phần ứng dụng dùng **TypeScript** [6] với cấu hình `"strict": true`; script kiểm chứng dùng JavaScript. Các kiểu `Movie`, `MovieDetails`, `MovieCredits` và `MovieVideosResponse` nằm trong `types/tmdb.d.ts`. Kiểm tra kiểu giúp phát hiện sai lệch khi biên dịch, không thay thế validation dữ liệu từ API hoặc cookie tại runtime.

1.7. Nền tảng dữ liệu mở The Movie Database (TMDB) API

The Movie Database (TMDB) [3] là cơ sở dữ liệu mở phổ biến nhất trên thế giới về phim điện ảnh và chương trình truyền hình. Hệ sinh thái MovieHub khai thác các nhóm dịch vụ REST API v3 của TMDB:

- **Nhóm dữ liệu trang chủ:** `/trending/movie/week`, `/movie/now_playing`, `/movie/top_rated` và `/discover/movie` cho phim kinh điển.
- **Nhóm dữ liệu chi tiết:** `/movie/{id}` cung cấp thông tin phim; các endpoint đuôi `/credits`, `/videos` và `/similar` lần lượt cung cấp diễn viên, video và phim tương tự.
- **Nhóm dữ liệu khám phá và lọc động:** `/discover/movie` dùng tham số thể loại, quốc gia, năm và cách sắp xếp. Với kiểu sắp xếp theo điểm, dịch vụ thêm ngưỡng số phiếu bình chọn.
- **Nhóm dữ liệu tìm kiếm thời gian thực:** `/search/movie` hỗ trợ tìm kiếm trực tiếp với Live Search Preview và trang `/search`.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Khảo sát và đặc tả yêu cầu hệ thống

2.1.1. Yêu cầu chức năng (Functional Requirements)

Hệ thống MovieHub được thiết kế để mang đến trải nghiệm khám phá điện ảnh tương đương các nền tảng thương mại hàng đầu như Netflix, Disney+. Bảng 2.1 tổng hợp đặc tả chi tiết 6 nhóm yêu cầu chức năng cốt lõi.

Bảng 2.1. Đặc tả các yêu cầu chức năng chính của hệ thống MovieHub (Nguồn: Nhóm tác giả đề xuất)

Mã YC	Tên chức năng	Mô tả chi tiết yêu cầu nghiệp vụ
FR01	Duyệt phim Trang chủ	Hero Banner tràn viền, nút xem trailer và các hàng phim điện ảnh chuyên đề: thịnh hành, đánh giá cao, đang chiếu rạp và phim kinh điển. `MovieRow` hỗ trợ cuộn ngang vô tận và hàng TOP 10 với số thứ tự phóng to.
FR02	Tìm kiếm thời gian thực	Thanh tìm kiếm trên Header hỗ trợ Live Search Preview gợi ý tối đa 5 kết quả rút gọn ngay khi người dùng gõ ký tự thông qua Route Handler `/api/search`; đồng thời cung cấp trang tìm kiếm độc lập `/search` có cơ chế Debounce.
FR03	Khám phá đa tiêu chí (`/browse`)	`FilterBar` hỗ trợ lọc đa chiều theo thể loại, quốc gia, năm phát hành và cách sắp xếp. Phân trang dùng trực tiếp trang TMDB, tối đa 20 phim/trang và giới hạn 500 trang.
FR04	Xem chi tiết phim (`/movie/[id]`)	Hiển thị thông tin tổng quan, thời lượng, năm phát hành, điểm đánh giá TMDB (`vote_average`), dàn diễn viên chính với slider ảnh đại diện tròn, trình phát Trailer YouTube chuẩn điện ảnh, và danh sách phim tương tự đề xuất.
FR05	Quản lý Danh sách yêu thích	Cho phép người dùng đánh dấu hoặc xóa phim khỏi danh sách Watchlist cá nhân mà không cần tài khoản, lưu trữ bền vững qua Cookie `HttpOnly` bảo mật.
FR06	Khám phá Diễn viên & Phim tương tự (`/movie/[id]`)	Tích hợp phân hệ hiển thị dàn diễn viên chính (Cast List) và danh sách phim điện ảnh tương tự đề xuất (Similar Movies) được phân tầng tải dữ liệu độc lập qua React Suspense Streaming.

2.1.2. Yêu cầu phi chức năng (Non-Functional Requirements)

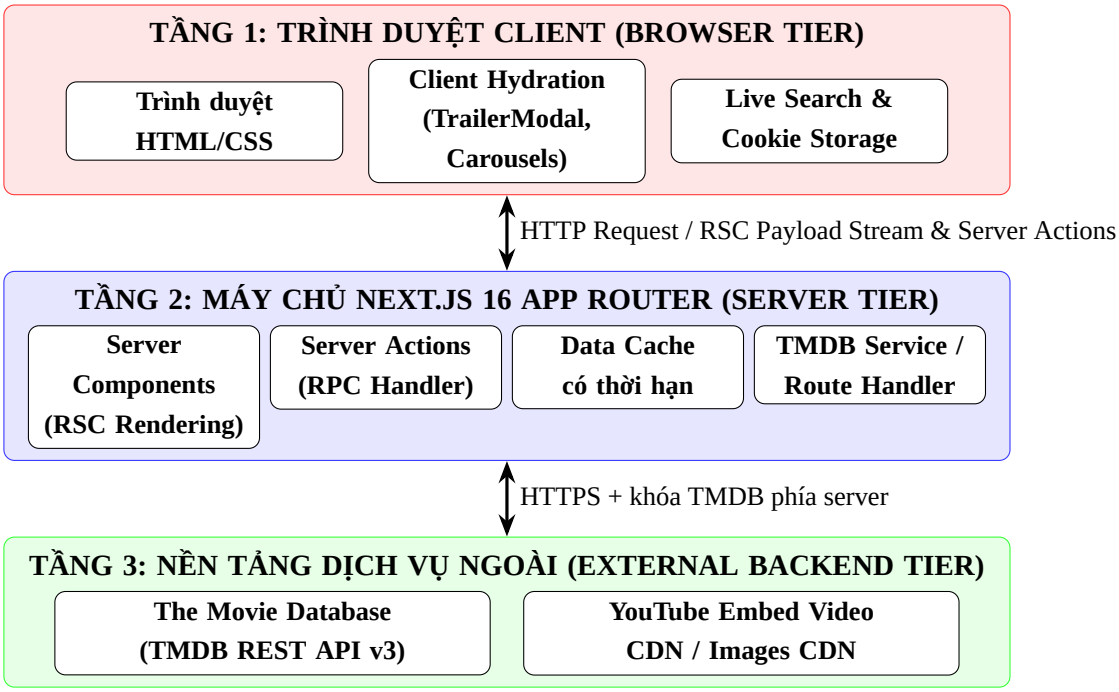
Các tiêu chí thiết kế và khả năng kiểm chứng hiện tại gồm:

- **NFR01 – Hiệu năng:** Ưu tiên Server Components, cache theo thời hạn và ảnh đáp ứng kích thước màn hình. Chưa có dữ liệu Lighthouse để xác nhận ngưỡng LCP.
- **NFR02 – Bố cục:** Khung poster xác định tỉ lệ trước khi tải ảnh; chưa có trace CLS trên trình duyệt để khẳng định giá trị bằng không.
- **NFR03 – Khóa API:** `TMDB_API_KEY` nằm trong biến môi trường phía server; không dùng tiền tố `NEXT_PUBLIC_` và không chuyển khóa qua props.
- **NFR04 – Chịu lỗi:** Tầng dịch vụ phân biệt HTTP 404 với lỗi timeout/mạng/cấu hình. Trang chủ truyền lỗi tải danh mục tới error boundary; nhánh phụ và gợi ý tìm kiếm có thông báo lỗi; trang chi tiết đưa lỗi dịch vụ đến error boundary có nút thử lại. Bản mã nguồn hiện tại không thay đổi DNS toàn cục; chưa có thử nghiệm mạng có kiểm soát.
- **NFR05 – Metadata:** Root layout cung cấp metadata chung; trang chi tiết dùng `generateMetadata` theo phim và ảnh backdrop khi có dữ liệu.

2.2. Kiến trúc hệ thống tổng thể

2.2.1. Mô hình kiến trúc 3 tầng phân lớp

Hệ thống MovieHub được thiết kế theo kiến trúc 3 tầng phân lớp hiện đại (3-Tier Modern Architecture), tận dụng tối đa sức mạnh của Next.js làm tầng trung gian điều phối thông minh. Sơ đồ kiến trúc được minh họa trong Hình 2.1.



Hình 2.1. Sơ đồ kiến trúc 3 tầng phân lớp hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế)

2.2.2. Cấu trúc thư mục dự án theo chuẩn App Router

Mã nguồn dự án được tổ chức khoa học, tách bạch rõ ràng giữa các trang định tuyến, component dùng chung, tầng dịch vụ và định nghĩa kiểu dữ liệu:

- `app/`: Chứa toàn bộ các tuyến đường định tuyến theo cấu trúc thư mục (File-system Routing):
  - `layout.tsx`: Root Layout chứa khung Header và Footer dùng chung toàn ứng dụng.
  - `page.tsx`: Trang chủ hiển thị Hero Banner và các hàng phim chủ đề.
  - `movie/[id]/page.tsx`: Tuyến đường động hiển thị thông tin chi tiết một bộ phim.
  - `browse/page.tsx`: Trang khám phá phim với bộ lọc đa chiều và phân trang số.
  - `search/page.tsx`: Trang kết quả tìm kiếm phim có hỗ trợ bộ lọc.
  - `watchlist/page.tsx`: Trang danh sách phim yêu thích cá nhân của người dùng.
  - `error.tsx`, `not-found.tsx`, `global-error.tsx`: Các trang bẫy lỗi và 404 tầng Root chuẩn phong cách Netflix Dark Cinema.

  - `sitemap.ts`, `robots.ts`: Tệp siêu dữ liệu SEO chuẩn Metadata File Conventions.
  - `actions/watchlist.ts`: Server Action ghi cookie; `api/search/route.ts`: Route Handler GET phục vụ Live Search.
- `components/`: Chứa các UI components được phân tách ranh giới rõ ràng theo hai thư mục con:
  - `components/client/`: Chứa các Client Component có tính tương tác cao (`"use client"`): `Header.tsx`, `NavbarDropdown.tsx`, `NavSearchBar.tsx` (thanh điều hướng và Live Search); `MovieRow.tsx`, `CastSlider.tsx`, `SimilarMoviesSlider.tsx` (các thanh cuộn phim và diễn viên); `FilterBar.tsx` (thanh lọc đa chiều); `TrailerModal.tsx` (dialog xem trailer); `WatchlistButton.tsx`, `ClearWatchlistButton.tsx` (tương tác danh sách yêu thích).
  - `components/server/`: Chứa các Server Component kết xuất trực tiếp trên máy chủ: `HeroBanner.tsx` (billboard banner), `MovieCard.tsx` (thẻ áp phích phim), `CastList.tsx` (danh sách diễn viên), `SimilarMovies.tsx` (danh sách phim tương tự), `Footer.tsx` (chân trang), cùng các skeleton loading trạng thái chờ (`CastSkeleton.tsx`, `MovieCardSkeleton.tsx`).
- `services/client.ts`: Hàm `fetchTMDB`, cấu hình Data Cache và khử trùng lặp; `movies.ts`, `discover.ts`, `search.ts`, `videos.ts` chứa các lời gọi theo nghiệp vụ; `tmdb.ts` tái xuất các hàm cho trang.
- `types/`: Chứa toàn bộ các định nghĩa kiểu dữ liệu TypeScript.

2.3. Thiết kế tích hợp dịch vụ và luồng dữ liệu

2.3.1. Kiến trúc tầng dịch vụ `services/client.ts`

Tất cả các lời gọi mạng ra ngoài TMDB đều được tập trung vào hàm cốt lõi `fetchTMDB<T>()`. Hàm này chịu trách nhiệm:

1. Dùng URL mặc định `https://api.themoviedb.org/3`; có thể thay bằng `TMDB_BASE_URL` để kiểm thử.
2. Đính kèm `api_key=${TMDB_API_KEY}` ở yêu cầu từ server tới TMDB; không dùng tiền tố `NEXT_PUBLIC_`. Cần tránh ghi URL có khóa vào log hoặc chuyển nó qua props.

3. Kiểm soát lỗi mạng và mã trạng thái HTTP, trả về thông báo lỗi chuẩn xác hoặc kích hoạt cơ chế dự phòng khi có sự cố.

2.3.2. Xử lý lỗi mạng và phạm vi DNS

Mã nguồn hiện tại sử dụng cơ chế phân giải tên miền mặc định của Node.js, không ghi đè `dns.lookup` và không cài Resolver DNS toàn cục. `fetchTMDB` kiểm tra HTTP status; các hàm nghiệp vụ xử lý lỗi tùy ngữ cảnh. Việc bỏ hook tránh tác động đến các yêu cầu khác trong cùng tiến trình. Dự án chưa có phép đo mạng có kiểm soát để kết luận nguyên nhân lỗi DNS hay khả năng tự phục hồi.

2.3.3. Phân tầng chiến lược Caching và Revalidation

Bảng 2.2 ghi thời hạn `next.revalidate` trong các hàm dịch vụ; đây là cấu hình Data Cache cho yêu cầu mạng, không mặc nhiên là thời hạn ISR của mọi route.

Bảng 2.2. Thời hạn revalidate của các yêu cầu TMDB trong tầng services

Nhóm dữ liệu	**Chiến lược Cache**	**Thời gian TTL**	**Mục đích và Lý do thiết kế**
Phim thịnh hành trong tuần	`fetch` Data Cache	3.600s	`getTrendingMovies` cập nhật định kỳ mỗi giờ.
Phim đang chiếu / điểm cao	`fetch` Data Cache	1.800s / 86.400s	`getNowPlayingMovies`, `getTopRatedMovies`.
Chi tiết, diễn viên, phim tương tự	`fetch` Data Cache	3.600s	Route chi tiết vẫn động vì đọc cookie; các phần phụ dùng Suspense.
Khám phá, kinh điển / tìm kiếm	`fetch` Data Cache	1.800s / 300s / 120s	`discoverMovies` (1.800s cho khám phá và phim kinh điển), `searchMovies` (300s cho trang tìm kiếm) và `searchQuickLive` (120s cho Live Search).

2.4. Thiết kế giao diện và trải nghiệm người dùng (UI/UX)

2.4.1. Hệ thống thiết kế chuẩn điện ảnh (Netflix Dark Cinema Design System)

Để mang lại trải nghiệm xem phim đẳng cấp và chân thực, giao diện MovieHub được xây dựng tuân thủ nghiêm ngặt theo hợp đồng thiết kế `DESIGN.md`. Bảng 2.3 liệt kê các thông số thiết kế chủ đạo.

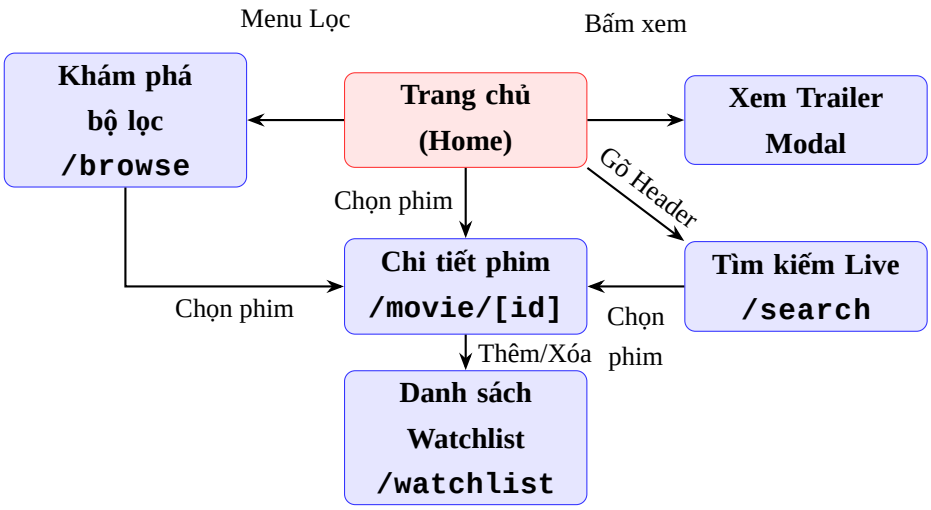
Bảng 2.3. Bảng mã màu và thông số thiết kế Netflix Dark Cinema (Nguồn: Trích xuất từ tài liệu DESIGN.md của dự án)

Yếu tố thiết kế	Giá trị mã màu	Mục đích và Vị trí áp dụng
Màu nền chủ đạo	`#141414`	Tạo chiều sâu điện ảnh tối đa, làm nổi bật màu sắc rực rỡ của các áp phích phim; giảm mỏi mắt cho người dùng trong điều kiện thiếu sáng.
Màu đỏ thương hiệu	`#E50914`	Điểm nhấn thị giác cho Logo thương hiệu, các nút kêu gọi hành động (CTA), viền active của danh mục và chỉ báo đang xem.
Màu bề mặt phụ	`#2F2F2F`	Sử dụng cho thanh điều hướng phụ, các thẻ card khi hover và nền khung skeleton chờ.
Màu văn bản chính	`#FFFFFF`	Chữ trắng trên nền xám đen (`#141414`) đạt chuẩn tương phản **WCAG 2.1 AA** với tỉ lệ xấp xỉ **18,4:1** (vượt xa ngưỡng tối thiểu 4,5:1 cho văn bản thường và 3,0:1 cho văn bản lớn). Đây là kiểm định tương phản cặp màu, không thay thế chứng nhận toàn trang.
Màu văn bản phụ	`#AAAAAA`	Sử dụng cho ngày phát hành, thông số thời lượng, tóm tắt nội dung phụ.

2.4.2. Sơ đồ điều hướng và luồng trải nghiệm người dùng

Hình 2.2 thể hiện sơ đồ luồng người dùng (User Navigation Flow) theo định hướng điện ảnh chuyên sâu (Movie-Only Cinema Flow). Kiến trúc điều hướng tinh giản triệt để phân hệ TV Series nhằm loại bỏ hoàn toàn các liên kết điều hướng gãy dẫn tới lỗi TMDB HTTP 404, tập trung 100% tài nguyên và luồng trải nghiệm vào thế

giới phim chiếu rạp từ Trang chủ, Khám phá thể loại (`/browse`), Chi tiết phim (`/movie/[id]`) đến Quản lý Watchlist cá nhân.



Hình 2.2. Sơ đồ luồng điều hướng trải nghiệm người dùng trong hệ thống MovieHub (Nguồn: Nhóm tác giả thiết kế)

2.5. Môi trường và công cụ phát triển

Dự án được triển khai trên nền tảng phần mềm hiện đại và ổn định:

- **Môi trường kiểm chứng (Runtime):** Node.js từ phiên bản 24; bộ minh chứng SP07 tại Bảng 4.1 ghi Node.js `v26.12.1` trên Linux. Dự án dùng API web stream và cơ chế phân giải tên miền mặc định của Node.js.
- **Khung ứng dụng (Framework):** Next.js phiên bản 16.4.5 (App Router) kết hợp thư viện React 19.2.8.
- **Ngôn ngữ và Công cụ quản lý gói:** TypeScript 5.x và npm (Node Package Manager).
- **Môi trường lập trình:** Visual Studio Code tích hợp tiện ích mở rộng ESLint, Tailwind CSS IntelliSense và GitLens.
- **Công cụ kiểm chứng:** ESLint, TypeScript, Node test runner và `next build`; dự án có cấu hình analyzer nhưng chưa lưu kết quả đo.
- **Công cụ biên soạn tài liệu kỹ thuật:** MiKTeX hoặc TeX Live có `XeLaTeX`; `latexmk` tự chạy lại để cập nhật mục lục và tham chiếu.

CHƯƠNG 3. HIỆN THỰC HỆ THỐNG

3.1. Cài đặt và cấu hình dự án

Chạy lại từ source MovieHub bàn giao, tại thư mục chứa `package.json`; không chạy `create-next-app@latest` để tạo dự án khác. Cần Node.js từ 24, npm, Git và Internet để cài dependency, tải Google Fonts khi build. Giữ `package-lock.json` để cài đúng bộ dependency:

```
node --version
npm --version
git rev-parse HEAD
npm ci
```

Sao chép `.env.example` thành `.env.local` bằng trình soạn thảo hoặc công cụ quản lý tệp, rồi điền khóa TMDB API v3 của người chạy. Các giá trị dưới đây là mẫu cấu hình, không phải khóa thật:

```
TMDB_API_KEY=your_tmdb_api_key_here
TMDB_BASE_URL=https://api.themoviedb.org/3
NEXT_PUBLIC_SITE_URL=http://localhost:3000
```

`TMDB_API_KEY` cần khi chạy ứng dụng với TMDB thật; khóa không có tiền tố `NEXT_PUBLIC_`. `TMDB_BASE_URL` mặc định là API v3 của TMDB; `NEXT_PUBLIC_SITE_URL` dùng cho metadata, sitemap và robots, đổi thành URL triển khai khi đưa ứng dụng lên mạng. Không đưa khóa vào client props, URL của trang web hoặc bài nộp; không nộp `.env.local`.

Thông tin phiên bản các gói phần mềm cốt lõi được cấu hình tường minh trong `package.json` của dự án:

```
{
  "name": "modern-programming-technologies",
  "version": "0.1.0",
  "dependencies": {
    "next": "16.4.5",
    "react": "19.2.8",
    "react-dom": "19.2.8"
  },
  "devDependencies": {
    "@next/bundle-analyzer": "16.4.5",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
```

```
}
```

Tệp `package.json` và `package-lock.json` xác nhận phiên bản `Next.js 16.4.5`; phiên bản `React v19.2.8` được xác nhận từ `package.json` và `npm ls react --depth=0`.

Lệnh chạy và kiểm tra. Với cấu hình TMDB thật, chọn một trong hai cách sau, không chạy đồng thời trên cùng cổng:

```
# Development: http://localhost:3000
npm run dev

# Production: build first, then start
npm run build
npm run start
```

Build dùng Webpack và không gọi TMDB vì các route liên quan đợi request bằng `connection()`; ứng dụng cần khóa và kết nối TMDB khi phục vụ request. Các lệnh kiểm tra là `npm run lint`, `npx tsc --noEmit` và `npm test`. Lệnh `npm run lab` tự build/start production với mock loopback, không cần khóa thật hoặc Chrome. Lệnh `npm run evidence` kiểm tra toàn sản phẩm, có thêm Watchlist và Chrome/Chromium; đặt `CHROME_BIN` tới executable nếu không được tự nhận diện. Cả hai cần quyền mở HTTP loopback và Internet tải font. Biến `MOVIEHUB_EVIDENCE_DIR` cho phép chọn nơi lưu log; mặc định lab tạo thư mục riêng mỗi lượt, còn evidence ghi vào `baocao/evidence/`.

3.1.1. Mô hình dữ liệu và phần không áp dụng

MovieHub không vận hành cơ sở dữ liệu riêng, không có ORM, model huấn luyện hay chỉ mục tìm kiếm cục bộ; vì vậy **migration, schema DB, seed DB và model/index không áp dụng** cho bản bàn giao. Dữ liệu phim, diễn viên và video được lấy từ TMDB API v3; các kiểu `Movie`, `MovieDetails`, `MovieCredits` và `MovieVideosResponse` được khai trong `types/tmdb.d.ts`. Đây là hợp đồng kiểu TypeScript, không phải schema cơ sở dữ liệu hay bộ kiểm tra JSON tại runtime.

Watchlist lưu mảng JSON các ID phim trong cookie `mh_watchlist`, tối đa 200 ID và 3800 byte tính cả tên cùng giá trị đã mã hóa, thời hạn 30 ngày. Server kiểm tra ID nguyên dương an toàn, loại trùng và xử lý cookie hỏng; người dùng không cần

tài khoản. Cookie có thể bị xóa hoặc hết hạn và không đồng bộ đa thiết bị. Dữ liệu mẫu chạy lại nằm trong `scripts/mock-tmdb-evidence.mjs`: fixture phim, credits, similar và video, bộ đếm `/_counts`, độ trễ có kiểm soát và nội dung đổi phiên bản để kiểm chứng cache. Không cần nhập dữ liệu hay dùng khóa TMDB thật cho lab.

3.2. Hiện thực Tầng 1A: Các tính năng cốt lõi bắt buộc

3.2.1. Phân định ranh giới Server/Client Component

Ứng dụng thực hiện phân tách ranh giới rõ ràng:

- **Root Layout (`app/layout.tsx`):** Đóng vai trò là Server Component gốc, chịu trách nhiệm nhập bộ font chữ chuẩn, cấu hình thẻ `<html>`, `<body>` và bao bọc toàn bộ khung ứng dụng bằng thanh điều hướng `Header` và chân trang `Footer`.
- **Trang chủ (`app/page.tsx`):** Đóng vai trò là Server Component điều phối, chịu trách nhiệm thu thập toàn bộ các danh sách phim song song và truyền dữ liệu dạng props tĩnh xuống các component con.

3.2.2. Server-side Data Fetching và song song hóa với `Promise.all`

Trang chủ `app/page.tsx` là Server Component bất đồng bộ. Bốn lời gọi dữ liệu được khởi tạo cùng lúc; kết quả được truyền vào giao diện sau khi tất cả thành công. Trang gọi `await connection()` trước khi fetch để chỉ render khi có request, giúp build không phụ thuộc TMDB. Lỗi tải dữ liệu được chuyển tới error boundary, không bị biến thành danh mục rỗng. Các fetch vẫn dùng Data Cache có thời hạn; HTML toàn trang không được ISR. Dữ liệu TMDB được lấy bởi `fetchTMDB` trong `services/client.ts`, sử dụng biến môi trường `TMDB_API_KEY` và `next: { revalidate }`. Khóa API không có tiền tố `NEXT_PUBLIC_`.

```
// Trich tu app/page.tsx
await connection();
const [trending, nowPlaying, topRated, classicData] =
  await Promise.all([
    getTrendingMovies(),
    getNowPlayingMovies(),
    getTopRatedMovies(),
    discoverMovies({ genre: "classic" }),
  ]);
```

Thời gian phản hồi thực tế phụ thuộc mạng và trạng thái cache; báo cáo không suy ra một giá trị mili giây cố định từ cấu trúc `Promise.all`. Các `MovieCard` được tạo trong trang server và truyền vào `MovieRow` bằng prop `cards`; mã của thanh cuộn vẫn ở client để xử lý thao tác nút mũi tên.

3.2.3. Dynamic Routing (`/movie/[id]`) và Dynamic SEO Metadata

Kể từ Next.js 15 và bắt buộc trong Next.js 16, tham số định tuyến động `params` đã chuyển thành Promise bất đồng bộ trên cả **Page Component**, **Hàm sinh Metadata**, và **Route Handler**:

```
// app/movie/[id]/page.tsx
interface PageProps {
  params: Promise<{ id: string }>;
}

// Cu phap ap dung tuong tu cho Route Handler dong:
// export async function GET(req: Request, { params }: { params: Promise
    <{ id: string }> })

export async function generateMetadata({ params }: PageProps): Promise<
    Metadata> {
  const { id } = await params;
    const movie = await getMovieDetails(id);
    const year = movie.release_date ? movie.release_date.split("-")[0] :
    "";

  return {
    title: `${movie.title} ${year ? `(${year})` : ""} | MovieHub`,
    description: movie.overview || `Thong tin phim ${movie.title}`,
    openGraph: {
      title: movie.title,
      description: movie.overview,
      images: movie.poster_path
        ? [`https://image.tmdb.org/t/p/w500${movie.poster_path}`]
        : [],
    },
  };
}
```

3.3. Hiện thực Tầng 1B: Các kỹ thuật chuyên sâu và nâng cao

3.3.1. Server Action cho mutation và Route Handler cho tìm kiếm

Tác vụ thay đổi dữ liệu dùng `toggleWatchlist` và `clearAllWatchlist` trong `app/actions/watchlist.ts`. Action kiểm tra ID phim, ghi cookie `HttpOnly`, `SameSite=Lax`, bật `Secure` trong production rồi gọi `revalidatePath` cho trang Watchlist.

Live Search là tác vụ đọc. `NavSearchBar` đợi 300ms, gọi GET `/api/search` và hủy yêu cầu cũ bằng `AbortController`. Route Handler trong `app/api/search/route.ts` xác thực từ khóa rồi gọi `searchQuickLive` trên server. Header không gọi Server Action cho từng phím. Tiêu đề `s-maxage=120` hướng tới shared cache/CDN; mức cache thực tế cần đo trên môi trường triển khai.

3.3.2. Streaming Suspense và Skeleton Loading States

Tại `app/movie/[id]/page.tsx`, thông tin chính cần `getMovieDetails`, `getMovieVideos` và trạng thái watchlist trước khi phần thân trang được dựng. Hai khối dữ liệu phụ `CastList` và `SimilarMovies` nằm trong các ranh giới Suspense độc lập với fallback tương ứng. Mã thật sử dụng `CastSkeleton` và một dãy khung poster chờ:

```
<Suspense fallback={<CastSkeleton />}>
  <CastList movieId={id} />
</Suspense>
<Suspense fallback={<div className="flex gap-3 overflow-hidden">
  {Array.from({ length: 6 }).map((_, i) => (
    <div key={i} className="w-36 aspect-[2/3] bg-zinc-800" />
  ))}
</div>}>
  <SimilarMovies movieId={id} />
</Suspense>
```

Đoạn mã rút gọn lớp CSS của fallback thứ hai nhưng giữ cấu trúc JSX thực tế. Suspense cho phép gửi phần đã sẵn sàng trước dữ liệu phụ khi truy vấn phụ chậm; thời điểm byte đầu và thời điểm vẽ nội dung cần đo riêng, không thể suy ra từ cấu trúc JSX.

3.3.3. Tối ưu hóa hình ảnh với `next/image` và YouTube Player API

Các ảnh phim trong các component chính dùng `next/image`:

- `MovieCard` khai báo kích thước ảnh theo breakpoint dưới đây; Hero Banner dùng `sizes="100vw"`:

```
sizes="(max-width: 640px) 40vw, (max-width: 1024px) 25vw, 15vw"
```

- Khung poster có tỉ lệ 2:3 để dành chỗ trước khi ảnh tải xong; hiệu quả CLS thực tế cần đo trên trình duyệt.
- `TrailerModal` nhúng YouTube bằng `iframe` với `autoplay=1`; khả năng tự phát còn phụ thuộc chính sách của trình duyệt.

3.4. Hiện thực các module giao diện người dùng chính

3.4.1. Module Hero Banner & Trailer Modal xem trailer tức thì

Component `TrailerModal.tsx` là Client Component, dùng phần tử HTML `dialog` và `showModal()` để trình duyệt quản lý focus và chặn tương tác ngoài modal. Khi người dùng bấm "Xem Trailer", modal mở trên giao diện; phím `Escape` hoặc nhấp bên ngoài đóng modal. Nếu TMDB không có trailer, component dùng liên kết tìm kiếm YouTube. Hàm chọn trailer nằm trong `services/videos.ts`.

3.4.2. Module MovieRow Slider & Carousel cuộn vô tận

Component `MovieRow.tsx` mang lại trải nghiệm cuộn phim đặc trưng của Netflix:

- Tích hợp hai nút điều hướng mũi tên trái và phải, tự động ẩn hiện thông minh khi người dùng rê chuột vào hàng phim.
- Áp dụng thuật toán kiểm tra ngưỡng biên cuộn (`scrollLeft` và `maxScrollLeft`): Khi cuộn đến gần cuối danh sách (ngưỡng 80px), hệ thống tự động cuộn mượt trở về đầu và ngược lại, tạo cảm giác danh sách lặp vòng vô tận (Infinite Loop) không giới hạn.
- Hỗ trợ thao tác vuốt cảm ứng (touch swipe) tự nhiên trên các thiết bị máy tính bảng và điện thoại di động.

3.4.3. Module FilterBar & Trang Khám phá `/browse`

Trang `/browse` cung cấp bộ công cụ tìm kiếm điện ảnh chuyên sâu:

- Thanh công cụ `FilterBar` cho phép kết hợp đồng thời các tiêu chí điện ảnh độc lập: (1) Thể loại (`genre`: 19 thể loại phim TMDB), (2) Quốc gia (`country`: 12 quốc gia và khu vực chọn lọc: Âu Mỹ, Hàn Quốc, Trung Quốc, Nhật Bản, Việt Nam...), (3) Năm phát hành (`year`: từ 2026 trở về trước), và (4) Sắp xếp (`sortBy`: độ phổ biến, điểm số, ngày ra mắt, doanh thu).
- **Phân trang:** `services/discover.ts` lấy đúng một trang TMDB, tối đa 20 phim và giới hạn 500 trang; chỉ loại ID trùng trong trang đó. TMDB có thể đổi thứ tự giữa các lượt tải, nên không cam kết danh sách bất biến hoặc không trùng xuyên trang.

Ở `/search`, tìm theo từ khóa chỉ hỗ trợ thể loại (kể cả kinh điển), năm, thứ tự liên quan, điểm số và ngày phát hành trong tối đa hai trang TMDB đầu tiên; hiển thị tối đa 24 phim. Quốc gia và doanh thu không có trong dữ liệu search nên được ẩn; URL cũ có các lựa chọn này chuyển về URL phù hợp. Query lặp được lấy giá trị đầu, loại ký tự điều khiển và giới hạn 100 ký tự trước khi xử lý; các bộ lọc được kiểm tra theo danh sách cho phép. Watchlist chỉ bỏ qua phim đã bị TMDB xóa (404); lỗi mạng hoặc dịch vụ được chuyển tới error boundary để tránh báo danh sách rỗng sai.

3.4.4. Module Thanh điều hướng nâng cao (`NavbarDropdown`, `NavSearchBar`)

Thanh điều hướng `Header` sở hữu các tính năng cao cấp:

- **Hiệu ứng thanh điều hướng:** Ở các trang có Hero Banner, Header bắt đầu trong suốt và chuyển sang nền `#141414` khi `window.scrollY > 30`.
- **Mega Menu Dropdown (`NavbarDropdown.tsx`):** Hiển thị danh sách đầy đủ 19 thể loại phim TMDB dưới dạng lưới 2 cột tiện lợi.
- **Live Search Bar (`NavSearchBar.tsx`):** Hộp tìm kiếm nhanh hiển thị kết quả xem trước ngay bên dưới thanh điều hướng, cho phép nhấp vào để chuyển thẳng tới trang chi tiết phim.

3.4.5. Module Khám phá Diễn viên và Tuyển tập Phim tương tự

Tại trang chi tiết phim (`/movie/[id]`), các component `CastList` và `SimilarMovies` cung cấp cái nhìn toàn diện về đội ngũ diễn viên chính và các tác phẩm điện ảnh cùng phong cách, được bọc trong các ranh giới React Suspense độc lập giúp tối ưu hóa luồng tải trang.

3.5. Hiện thực cơ chế tự phục hồi và khử trùng lặp dữ liệu

3.5.1. Giới hạn xử lý hạ tầng

Không sử dụng DNS hook trong bản mã nguồn hiện tại. Lỗi DNS cần được chẩn đoán ở môi trường triển khai; việc thay DNS không phải tính năng hay tối ưu của Next.js. Test kiến trúc kiểm tra tầng dịch vụ không ghi đè `dns.lookup`.

3.5.2. Thuật toán khử trùng lặp phim (`deduplicateMovies`)

Khi tìm kiếm gom nhiều trang TMDB hoặc upstream có ID trùng, kết quả cần loại bản sao trong tập dữ liệu đã lấy. Hàm `deduplicateMovies` trong `services/client.ts` dùng `Set` để giữ phần tử đầu theo ID; test tương ứng nằm trong `tests/deduplicate.test.mjs`:

```
function deduplicateMovies(movies: Movie[]): Movie[] {
  const seen = new Set<number>();
  const unique: Movie[] = [];
  for (const movie of movies) {
    if (movie && movie.id && !seen.has(movie.id)) {
      seen.add(movie.id);
      unique.push(movie);
    }
  }
  return unique;
}
```

3.6. Tổ chức phát triển và tiến độ thực hiện

3.6.1. Bảng phân công trách nhiệm và minh chứng Git đối soát

Bảng 3.1 tổng hợp nhiệm vụ theo hồ sơ phân công và một số commit có thể đối chiếu. Không suy tỷ lệ đóng góp hoặc tác giả trực tiếp của mọi dòng mã từ số commit; phần việc tích hợp qua tài khoản khác cần thành viên xác nhận.

Bảng 3.1. Nhiệm vụ theo hồ sơ phân công và mốc Git đối chiếu

Họ và tên	Mã số SV	Nhiệm vụ kỹ thuật đảm nhiệm chính	Minh chứng Git / PR đối soát	Tỷ lệ
Nguyễn Công Quân (Trưởng nhóm)	3123560070	Khởi tạo App Router, tầng dịch vụ TMDB, trang chủ, `MovieRow`, `TrailerModal`, pipeline minh chứng, CI và tổng hợp báo cáo.	Mốc mã nguồn: `2b04d20`, `82c7e0e`, `6f5e663`, `3b86fa1`, `f25cf4b`. Đối chiếu tác giả và người tích hợp bằng `git show`.	Không định lượng
Nguyễn Quốc Đại	3123560013	Đối chiếu yêu cầu môn học, GET Route Handler cho tìm kiếm, xử lý lỗi tầng Root, SEO, Security Headers, test Node; loại bỏ DNS hook và route TV không hoàn chỉnh.	Mốc mã nguồn: `8fe18e6`, `8c059f7`, `953db35`, `80fb08e`; hồ sơ đối chiếu: `baocao/DANH_GIA_REQUIRE.md`.	Không định lượng
Nguyễn Thái Vinh	3123410433	Theo hồ sơ phân công: giao diện tìm kiếm, Watchlist, Browse/Filter và phân trang; bản cuối dùng thể loại, quốc gia, năm, thứ tự và trang theo ngữ cảnh.	Hồ sơ `THAI.md`; các nhánh tính năng tìm kiếm, Watchlist và điều hướng có trong lịch sử. Chưa xác nhận tác giả trực tiếp từng commit tích hợp.	Không định lượng

3.6.2. Tiến độ thực hiện theo tuần (Kế hoạch 9 tuần)

Kế hoạch học phần kéo dài 9 tuần. Bảng 3.2 đối chiếu nhóm công việc với hiện vật mã nguồn, không xác nhận công việc đã hoàn thành đúng tuần dự kiến. Nhật ký riêng nằm ở `individual/nhatky.tex`; `log.md` giữ diễn giải và kết quả lịch sử. Ngày ghi nhật ký không mặc nhiên là ngày tạo commit: ví dụ nhật ký ghi `953db35` ở ngày 05/11/2026, trong khi Git ghi commit ngày 19/10/2026. Chưa có xác nhận để coi hai ngày này là cùng một sự kiện; không dùng bảng kế hoạch để chứng minh ngày thực hiện, biên bản họp hay phần tự kiểm tra của từng thành viên.

Bảng 3.2. Các nhóm công việc trong kế hoạch 9 tuần và hiện vật tương ứng

Giai đoạn	Mục tiêu trọng tâm	Kết quả kỹ thuật nghiệm thu
Tuần 1	Khởi tạo và thiết lập kiến trúc	Có App Router, TypeScript strict, Tailwind CSS v4 và tầng dịch vụ TMDB trong `services/`.
Tuần 2	Giao diện cốt lõi & Data Fetching	Có Root Layout, Header và trang chủ với bốn nhóm dữ liệu gọi song song; ảnh chính dùng `next/image`.
Tuần 3	Dynamic Routing & Chi tiết phim	Hiện thực tuyến đường động `/movie/[id]`; xây dựng giao diện chi tiết, hiển thị backdrop, thông tin doanh thu, thời lượng; thiết lập hàm sinh siêu dữ liệu động `generateMetadata`.
Tuần 4	Kỹ thuật cốt lõi Streaming & Actions	Triển khai React Suspense Streaming cho dàn diễn viên và phim tương tự; xây dựng Server Actions quản lý Watchlist với Cookie `HttpOnly`; hoàn thiện bộ ba trang `loading`, `error`, `not-found`.
Tuần 5	Tìm kiếm Debounce & Bộ lọc	Xây dựng trang tìm kiếm `/search` có cơ chế Debounce; khởi dựng trang khám phá `/browse` kết nối endpoint TMDB Discover; hoàn thiện cơ chế cuộn mượt cho các hàng phim.
Tuần 6	Hoàn thiện tính năng nâng cao	Lập trình component `TrailerModal.tsx` phát YouTube trailer nổi; hoàn thiện bộ lọc `FilterBar` và bản phân trang ban đầu (bản cuối dùng trực tiếp trang TMDB tối đa 20 phim); xây dựng Mega Menu Dropdown và Live Search trên Header; rà soát lỗi mạng và loại bỏ hook DNS toàn cục.
Tuần 7	Kiểm thử tích hợp ban đầu	Chạy lint, type-check, các test Node và production build; tổng hợp ma trận yêu cầu sơ bộ.
Tuần 8	Rà soát 11 hạng mục, chốt pipeline kiểm chứng, tích hợp mock counter	Tiếp thu 11 nhận xét từ bảng đánh giá môn học; dọn dẹp mã chết (loại bỏ monkey-patch DNS, route `/tv` gãy); chuẩn hóa hệ thống bắt lỗi tầng Root (`error.tsx`, `global-error.tsx`, `not-found.tsx`) và tiêu đề bảo mật; tích hợp mock TMDB loopback có bộ đếm request `/_counts` để kiểm chứng quy luật `0 -> 1 -> 1 -> 2 -> 2`.
Tuần 9	Đồng bộ hóa báo cáo, hoàn thiện ma trận minh chứng, AI disclosure, phân công đóng góp	Hoàn thiện pipeline kiểm chứng tự động `npm run evidence`; cập nhật ma trận minh chứng đối soát yêu cầu; đồng bộ hóa thời gian streaming thực tế; hoàn thiện tuyên bố AI Disclosure và phân công đóng góp; biên dịch và chuẩn hóa báo cáo học thuật LaTeX đa nền tảng.

3.6.3. Quy trình làm việc nhóm với Git Flow

Dự án dùng Git và có lịch sử tích hợp qua Pull Request:

- Bản mã nguồn đang kiểm tra đã chạy production build. Chưa có bằng chứng trong repo về cấu hình bảo vệ nhánh `main` trên GitHub.
- Các nhánh tính năng trong lịch sử gồm nhóm trang chủ, trang chi tiết, tìm kiếm/Watchlist và Browse/Filter.
- Lịch sử Git tại mốc đối soát có 10 merge commit ghi “Merge pull request” (PR

#1–#10). Repo có workflow CI; chưa xác minh quy tắc review bắt buộc của từng PR trên GitHub.

CHƯƠNG 4. KIỂM THỬ VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Phương pháp và phân định hai cấu hình Build (Normal vs Mock Evidence)

Nhóm đánh giá hai loại bằng chứng: (1) mã nguồn và lệnh kiểm tra tĩnh có thể chạy lại; (2) kết quả vận hành cần môi trường và dữ liệu đo cụ thể. Các lệnh dùng cho bản mã nguồn hiện tại là `npm run lint`, `npx tsc --noEmit`, `npm test` và `npm run build`.

Mốc đối soát báo cáo: 07/11/2026. Source bàn giao tại Git HEAD `bd5b9b9` chứa bộ lab và minh chứng viết lại. Các lượt ngày 07/11 chạy khi HEAD còn là `7e06a9a` và `working_tree_dirty=true`; không đổi HEAD trong log thành commit bàn giao. SHA-256 của tập mã runtime/test/config hiện tại khớp hai lượt SP07 và LB07:

> `8a76a4540cc2ddf4e4d2450d86d9610841651afde68bce76ca4082dd17b18422`.

Fingerprint không bao gồm báo cáo, ảnh, nhật ký và tài liệu lab; nó không thay thế mã phiên bản của toàn bộ hồ sơ. Bảng 4.1 định danh từng bộ để tránh ghép kết quả khác lượt.

Bảng 4.1. Nguồn minh chứng và phạm vi sử dụng

Mã bộ	Thư mục và run ID	Trạng thái, cấu hình và phạm vi
SP07	`labs/streaming-cache/evidence/rebuilt-full/2026-11-07T09:13:45.024Z`	`passed`, `verification=full`; Node `v26.12.1`, Next `16.4.5`, Linux. 43 test, build/typecheck, HTTP, Watchlist và 16 kiểm tra Chrome.
LB07	`labs/streaming-cache/evidence/rebuilt-solution/2026-11-07T09:11:50.194Z`	`passed`, `verification=lab`; cùng phiên bản và fingerprint với SP07 nhưng khác lượt. Chỉ streaming/cache và kiểm tra nền.
ST07	`labs/streaming-cache/evidence/rebuilt-starter/2026-11-07T09:12:44.078Z`	`failed` đúng tại streaming; các bước trước đó đạt. Trang starter khác nên fingerprint khác. Không phải hồi quy sản phẩm.
CM07	`labs/streaming-cache/evidence/rebuilt-comment-solution/2026-11-07T09:13:25.173Z`	`passed`; lời giải giữ comment vẫn được chấp nhận. Không chạy Chrome, có fingerprint riêng.

Mã bộ	Thư mục và run ID	Trạng thái, cấu hình và phạm vi
LS05	`baocao/evidence/2026-11-05T10:36:52.952Z`	Minh chứng lịch sử: Next `16.3.6`, HEAD `7f23fab`; fingerprint khác. Không dùng để khẳng định kết quả của bản 16.4.5.

Đọc `mock-runtime.json` trong từng thư mục để lấy đầy đủ Git HEAD, `run_id`, `source_sha256`, phiên bản và trạng thái. ST07, CM07 cùng ghi HEAD `7e06a9a` và working tree có thay đổi, nhưng không có cùng source với SP07/LB07. Bộ kiểm tra luồng hướng dẫn nằm tại `labs/streaming-cache/evidence/rebuilt-checks/checks.json`; log cài `npm ci`, diff và patch roundtrip đi kèm. Không cộng 43 ca lặp ở nhiều lượt thành số test mới.

Để bảo đảm tính khách quan học thuật và khả năng kiểm chứng độc lập, hệ sinh thái MovieHub được thiết lập rõ ràng thành hai chế độ/cấu hình build tách biệt:

1. **Cấu hình Build Sản xuất Tiêu chuẩn (Normal Production Build):**
    - **Thư mục đầu ra:** Lưu trữ độc quyền tại `.next/`.
    - **Kết nối dữ liệu:** Trỏ trực tiếp tới TMDB API thật ngoài Internet (`api.themoviedb.org/3`).
    - **Thời hạn Data Cache (TTL):** Các fetch trang chủ có thời hạn thấp nhất 30 phút (1.800 giây); danh sách phim trong Sitemap dùng Data Cache 1 giờ (3.600 giây). Trang chủ và Sitemap gọi `connection()` để render khi có request, không gọi TMDB lúc build. Đây là SSR với Data Cache, không phải ISR HTML của toàn route. Khóa và kết nối TMDB cần khi ứng dụng phục vụ request; lỗi dịch vụ vẫn được xử lý theo từng ngữ cảnh.
    - **Minh chứng lưu trữ:** `baocao/evidence/build-output.txt` là log build thường lịch sử; phải đọc ngày và phiên bản riêng. Bảng route của bản 16.4.5 trong báo cáo lấy từ production mock SP07, không trình bày log thường cũ như một lượt build mới.
2. **Cấu hình Build Minh chứng Thực nghiệm (Mock Evidence Build):**
    - **Kích hoạt và Thư mục đầu ra:** Được kích hoạt khi đặt biến môi trường `MOVIEHUB_EVIDENCE_BUILD=1`, xuất kết quả vào thư mục cô lập `.next-evidence/`, hoàn toàn không làm sai lệch hay ghi đè thư mục `.next/` tiêu chuẩn.

- **Kết nối dữ liệu:** Trỏ tới máy chủ mock nội bộ trên loopback (`http://127.0.0.1:{PORT}/3`).
- **Trình biên dịch:** Cả build thường và build minh chứng dùng `next build --webpack`, lựa chọn được Next.js hỗ trợ để tránh lỗi cổng nội bộ của Turbopack trong môi trường hạn chế. Lint từ chối mọi warning; quy trình minh chứng kiểm tra log build có 0 error và 0 warning.
- **Thời hạn Data Cache rút ngắn:** Ép chu kỳ revalidate xuống 15 giây (`TMDB_MOCK_REVALIDATE_SECONDS=15`) để kiểm chứng vòng đời bộ đếm cache mà không phải chờ 30 phút. Ràng buộc an toàn (`services/client.ts`) bảo đảm cấu hình TTL 15s chỉ được phép thực thi khi host là loopback `127.0.0.1`; cấu hình này lập tức ném lỗi ngoại lệ nếu chạy với TMDB thật.
- **Minh chứng lưu trữ:** `build-mock-output.txt` thuộc SP07. Lượt LB07 phục vụ riêng bài học, không bao gồm Watchlist/Chrome.

Giới hạn về chỉ số hiệu năng. TTFB là thời điểm byte đầu tiên về client, còn FCP là thời điểm trình duyệt vẽ nội dung đầu tiên. Streaming/cache được đo qua HTTP; SP07 có thêm 16 kiểm tra tương tác Chrome trong `browser-interactions.json`. Đây là kiểm tra chức năng và bố cục theo kịch bản, không phải phép đo Core Web Vitals, Lighthouse hoặc RUM. Báo cáo không công bố LCP, CLS, INP, mức giảm bundle hay tỷ lệ tăng tốc khi chưa có dữ liệu gốc tương ứng.

4.2. Kiểm tra tự động và khả năng chạy lại

Bảng 4.2. Các kiểm tra có thể thực thi lại trên mã nguồn hiện tại

Lệnh	Phạm vi xác minh	Kết quả
`npm run lint`	Quy tắc ESLint của dự án Next.js.	Exit code 0
`npx tsc --noEmit`	Kiểu dữ liệu TypeScript ở chế độ `strict`.	Exit code 0
`npm test`	Toàn bộ 43 ca kiểm thử Node test runner về dữ liệu phim, ranh giới Server/Client, cache, tìm kiếm và Watchlist.	43/43 passed
`npm run evidence`	Lint, 43 test, build/typecheck, HTTP streaming/cache/Watchlist và 16 kiểm tra Chrome; bộ SP07.	Passed
`npm run lab`	Kiểm tra nền và HTTP streaming/cache; bộ LB07. Starter ST07 phải thất bại tại streaming.	LB07 passed

Kết quả bảng trên đọc từ SP07/LB07, không gán trạng thái của lượt này cho lượt khác. Kiểm tra đóng gói patch có một ca riêng, lệnh chạy ở mục 5.10; không nằm trong 43 ca của `npm test` và không dùng để chấm lời giải học viên.

Tệp `.github/workflows/check.yml` cấu hình `npm ci`, `npm audit --audit-level=moderate`, Chrome và `npm run evidence` khi push hoặc mở Pull Request; dùng mock nên không cần secret TMDB và lưu kết quả thành CI artifact. Đây là cấu hình đã có trong repo, **chưa xác nhận một lượt CI trên GitHub**. `tests/page-flows.test.mjs` gọi trang và Server Action thật với fixture để kiểm tra lỗi trang chủ, query lặp, bộ lọc, phân trang và Watchlist. HTTP production của SP07 nằm trong `routes-http.json` và `watchlist-http.json`; Chrome được ghi riêng trong `browser-interactions.json` cùng bộ.

4.3. Bằng chứng về mô hình render và cache

Bảng 4.3. Phân loại route từ build Webpack production mock SP07

Route	Kết quả	Giải thích từ mã nguồn
`/`	Động (SSR)	`connection()` đợi request; các fetch giữ Data Cache với thời hạn riêng, thấp nhất 1800 giây.
`/_not-found`	Tĩnh	Trang lỗi mặc định được tạo trước.
`/api/search`	Động	Route Handler đọc query từ request.
`/browse`	Động	Đọc `searchParams` để lọc.
`/movie/[id]`	Động	Đọc tham số `id` và cookie Watchlist; có hai ranh giới Suspense.
`/robots.txt`	Tĩnh	Tệp chỉ dẫn thu thập dữ liệu tìm kiếm sinh từ `app/robots.ts`.
`/search`	Động	Đọc `searchParams.q`.
`/sitemap.xml`	Động	`connection()` đợi request; danh sách phim dùng Data Cache 1 giờ.
`/watchlist`	Động	Đọc `await cookies()` theo người dùng.

Route phim không hợp lệ. Bộ SP07 gọi `/movie/not-a-number`, chấp nhận HTTP 200 hoặc 404 nhưng bắt buộc có giao diện không tìm thấy và `robots=noindex`; kết quả nằm trong `routes-http.json`. Với response streaming, Next.js có thể đã gửi mã HTTP 200 trước khi `notFound()` được kích hoạt; đây là "soft 404" theo tài liệu Next.js [1], không phải bằng chứng HTTP status 404. Nếu yêu cầu bắt buộc trả status 404, cần kiểm tra tài nguyên trước khi stream.

Ranh giới Server/Client tại trang chủ. `app/page.tsx` gọi TMDB qua tầng dịch vụ trên server, tạo `MovieCard` rồi truyền phần tử React qua prop `cards` cho `MovieRow`. Test `tests/server-client-boundary.test.mjs` kiểm tra slider không nhập `MovieCard` vào module graph client. SP07 xác nhận HTML server có đủ bốn tiêu đề hàng phim và không chứa chuỗi `api_key=`, ghi trong `routes-http.json`. Các kiểm tra này không thay thế kiểm tra toàn bộ lưu lượng mạng hoặc đo kích thước bundle.

4.3.1. Ranh giới module và giới hạn đo bundle

Cấu hình `@next/bundle-analyzer` cho phép chạy `npm run analyze`. Chưa có tệp phân tích gốc đi kèm lượt kiểm chứng hiện tại để công bố dung lượng theo

component hay mức giảm JavaScript. Test mã nguồn chỉ kiểm tra các slider client không import `MovieCard`; kết quả này không phải phép đo byte tải xuống, thời gian hydration hoặc thời gian vẽ.

4.3.2. Kiểm chứng Data Cache bằng bộ đếm origin

Lệnh `npm run evidence` tạo bản build production riêng trong thư mục cô lập `.next-evidence`. Script dùng mock TMDB trên HTTP loopback, khóa giả và TTL 15 giây; cấu hình `cacheComponents` ở trạng thái tắt (`false`), vận hành theo Previous Caching Model với `fetch` revalidate. TTL ngắn 15s chỉ được chấp nhận khi host là `127.0.0.1`; cấu hình này bị ném lỗi từ chối ngay lập tức nếu trỏ tới TMDB thật.

Bảng 4.4. Bộ đếm mock origin cho mỗi endpoint của một ID phim mới

Endpoint	Trước	Miss	Hit	Refresh	Hit mới
`/movie/{id}`	0	1	1	2	2
`/movie/{id}/videos`	0	1	1	2	2
`/movie/{id}/credits`	0	1	1	2	2
`/movie/{id}/similar`	0	1	1	2	2

Nguồn dữ liệu: `labs/streaming-cache/evidence/rebuilt-full/cache-mock.json`. Script gọi cùng route hai lần, đợi hết TTL, yêu cầu lại, đợi background refresh kết thúc rồi kiểm tra một cache hit mới. HTML phải đổi từ “Phiên bản 1” sang “Phiên bản 2”, chứng minh dữ liệu thay đổi sau revalidation cùng với bộ đếm origin. Bộ đếm tăng tại HTTP mock origin, không suy từ số lần gọi hàm ở page hoặc metadata. Kết quả xác nhận Data Cache ở bản production cục bộ với dữ liệu giả; không chứng minh shared cache tại CDN hay mức gia tốc TMDB thật. Độ trễ do mock chủ động thêm không được dùng làm điểm Lighthouse hoặc Core Web Vitals.

4.3.3. Streaming và Watchlist qua HTTP production

Streaming. `labs/streaming-cache/evidence/rebuilt-full/streaming-mock.json` ghi thời điểm, số byte từng chunk và lần đầu thấy phần chính, hai fallback cùng hai kết quả. Script yêu cầu phần chính và fallback nằm ở chunk trước kết quả tương ứng; kết quả phải đến ít nhất 1000 ms sau phần chính trong kịch bản mock này. Việc tách thành nhiều chunk nhưng gửi dồn cùng lúc cũng bị từ chối.

Các marker là thuộc tính HTML riêng biệt, tránh nhầm chuỗi tên diễn viên trong RSC payload với HTML đã được stream. Mock trì hoãn credits 2,5 giây và similar 4 giây. Đây là thời điểm nhận HTML ở HTTP client, không phải thời điểm trình duyệt đã vẽ; chưa có Network waterfall/video đi kèm.

Watchlist. `watchlist-http.json` thuộc SP07 kiểm tra POST Server Action thật, cờ `HttpOnly`, `Secure` và `SameSite=Lax`, thêm/xóa, tải lại bằng GET mới và phiên cookie độc lập. Không lưu giá trị cookie vào minh chứng. SP07 có thêm kiểm tra Chrome cho rollback và các tương tác được liệt kê trong `browser-interactions.json`. Các ảnh `baocao/evidence/watchlist-browser-saved.png` và `baocao/evidence/watchlist-browser-list.png` thuộc lượt lịch sử riêng; chúng không phải ảnh do SP07 tạo lại, không phải video hoặc Network waterfall.

Cookie đầu vào chỉ giữ ID số nguyên dương an toàn, loại trùng và giới hạn 200 mục; kiểm tra 3800 byte tính cả phần mã hóa giá trị cookie. Nút lưu dùng `useOptimistic` theo trạng thái server, khôi phục khi thao tác thất bại và hiển thị lỗi bằng `role="alert"`. Cookie là dữ liệu cục bộ theo trình duyệt, không có đăng nhập và không đồng bộ đa thiết bị.

Khả năng truy vết. Mỗi bộ trong Bảng 4.1 giữ run ID, ngày chạy, Git HEAD, working tree, phiên bản và SHA-256 nguồn. Chỉ kết luận đúng phạm vi khi `mock-runtime.json` của bộ đó có `status=passed`. ST07 cố ý `failed` ở streaming; không tính là bộ sản phẩm đạt. Không ghép artifact SP07, LB07, ST07, CM07 hay LS05 thành một lượt đo.

4.4. Ma trận đáp ứng yêu cầu môn học

Bảng 4.5. Bảng phủ yêu cầu, nguồn minh chứng và trạng thái

Tiêu chí / tầng	File hiện thực	Nguồn và minh chứng	Trạng thái
1A: App Router, page/layout	`app/layout.tsx`, `app/page.tsx`; `next/link` ở card.	SP07: `build-mock-output.txt`; Bảng 4.3.	Có mã; build đạt
1A: Server/Client và luồng render	`app/page.tsx`, `components/client/MovieRow.tsx`; slider nhận card từ server.	SP07: test `server-client-boundary`, `routes-http.json`; Bài 1 lab.	Đã kiểm chứng phạm vi nêu

Tiêu chí / tầng	File hiện thực	Nguồn và minh chứng	Trạng thái
1A: Fetch phía server	`services/client.ts`; trang gọi các hàm từ `services/tmdb.ts`.	SP07: `routes-http.json`, `streaming-mock.json`; không có khóa trong HTML kiểm tra.	Đạt trên mock production
1B: Route động và lỗi	`app/movie/[id]/page.tsx`, `loading.tsx`, `error.tsx`, `not-found.tsx`.	SP07: build, test và `routes-http.json`; ID sai có UI và `noindex`.	Đạt; không cam kết status 404 khi stream
1B: Cache và revalidation	`services/client.ts`, `next.config.ts`.	SP07: `cache-mock.json`; Bảng 4.4.	Đạt cục bộ; chưa đo CDN
1B: Mutation và API query	`app/actions/watchlist.ts`, `app/api/search/route.ts`.	SP07: `watchlist-http.json`, `routes-http.json`, `test-output.txt`.	Đã kiểm chứng
1B: Suspense, Image/Font, SEO	Trang phim, `MovieCard`, root layout, `sitemap.ts`, `robots.ts`.	SP07: streaming/build và `browser-interactions.json`; cấu hình ảnh ở `next.config.ts`.	Có hiện thực; chưa đo Web Vitals
2: Chạy lại và cấu hình	`README.md`, `.env.example`, `package-lock.json`.	Mục 3.1.; `rebuilt-checks/checks.json` và log cài dependency.	Đã chạy trên Linux
2: Dữ liệu / migration	`types/tmdb.d.ts`, cookie Watchlist, mock TMDB.	Mục 3.1.1.; không có DB riêng.	Migration và model/index không áp dụng
2: Kiểm thử	`tests/`, `labs/streaming-cache/check-starter.test.mjs`.	SP07: 43/43; patch roundtrip một ca riêng ở `rebuilt-checks/`.	Đạt; không cộng trùng
2: CI và Git	`.github/workflows/check.yml`; lịch sử Git.	Có workflow, commit và merge trong repo; Git HEAD gốc tại Bảng 4.1.	Chưa xác nhận CI GitHub hoặc luật review PR
2: Bảo mật, lỗi và tương tác	Route Handler, Watchlist, error boundary, `next.config.ts`.	SP07: test, HTTP và 16 ca Chrome gồm rollback, focus, bộ lọc và tìm kiếm.	Đạt kịch bản đã kiểm; không là kiểm toán bảo mật toàn diện

Tiêu chí / tầng	File hiện thực	Nguồn và minh chứng	Trạng thái
Hồ sơ: nhật ký / AI	`individual/nhatky.tex`, `log.md`; Phụ lục 4.	Git xác minh thay đổi mã; nguồn AI, ngày sự kiện và xác nhận thành viên có giới hạn riêng.	Đối chiếu một phần
D0 và lab thực hành	`labs/streaming-cache/`; Phụ lục 5.	ST07, LB07, CM07 và `rebuilt-checks/`; có mục tiêu, starter, bước làm, ba bài và đáp án.	Đủ nội dung; chưa thử thời lượng với nhóm khác
3: Hiệu năng nâng cao	Bộ đo cache/streaming và cấu hình analyzer.	SP07 kiểm tra cơ chế; chưa có dữ liệu gốc Lighthouse, bundle hay CDN.	Chưa xác nhận các chỉ số này

Mỗi mã bộ trỏ tới thư mục, commit gốc và metadata trong Bảng 4.1. “Có mã/cấu hình” không đồng nghĩa đã chạy CI hoặc đã được người học kiểm chứng. Ma trận không tự suy điểm D1–D3 từ ảnh yêu cầu lab, cũng không công bố mức tăng tốc khi chưa có phép đo đối chứng.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được. MovieHub minh họa được các cơ chế chính của Next.js 16 trên một nghiệp vụ đơn giản: App Router, Server Components lấy dữ liệu TMDB, Client Components cho thao tác giao diện, route động với tham số bất đồng bộ, Data Cache có thời hạn, Streaming Suspense, metadata động, Server Action cho Watchlist và Route Handler GET cho tìm kiếm. Bản build production phân loại trang chủ và Sitemap là động nhờ `connection()`; dữ liệu TMDB vẫn dùng Data Cache có thời hạn. Các route phụ thuộc query, ID hoặc cookie cũng là động. Lint, kiểm tra kiểu, các test tự động và build Webpack production được kiểm chứng bằng `npm run evidence`; kết quả và SHA-256 mã nguồn nằm trong `mock-runtime.json`. CI cấu hình chạy lại quy trình đó khi cập nhật repo.

2. Giới hạn được nhận diện. Việc truyền phần tử `MovieCard` từ server vào slider client làm rõ ranh giới module. SP07 kiểm chứng HTTP production và 16 kịch bản Chrome bằng dữ liệu giả có kiểm soát; LB07 dành cho lab. Chưa có phép đo bundle, Core Web Vitals, trace hoặc video Network waterfall. Watchlist có giới hạn cookie và không đồng bộ đa thiết bị. Chưa kiểm chứng PowerShell/Windows, thời lượng học 40 phút với nhóm khác hay một lượt CI GitHub.

3. Hướng hoàn thiện. Nhóm định hướng các bước phát triển tiếp theo: (1) bổ sung bộ kiểm thử E2E với Playwright để ghi nhận trace/video trực quan cho Network waterfall; (2) nâng cấp phân hệ Watchlist từ Cookie sang hệ quản trị cơ sở dữ liệu (PostgreSQL/Prisma) kèm cơ chế xác thực người dùng OAuth2/NextAuth để đồng bộ đa thiết bị; (3) đánh giá Cache Components trong một thử nghiệm riêng khi có nhu cầu; tính năng này chưa được bật trong dự án.

TÀI LIỆU THAM KHẢO

[1] Vercel Inc., “Next.js Documentation: App Router, Server Components and Data Fetching,” 2026. [Trực tuyến]. Địa chỉ: https://nextjs.org/docs. [Truy cập: 15-09-2026].

[2] React Core Team, “React 19 Documentation and React Server Components Specification,” Meta Platforms Inc., 2026. [Trực tuyến]. Địa chỉ: https://react.dev. [Truy cập: 18-09-2026].

[3] The Movie Database, “TMDB API Reference Documentation v3,” 2026. [Trực tuyến]. Địa chỉ: https://developer.themoviedb.org/docs. [Truy cập: 20-09-2026].

[4] Google Chrome Developers, “Web Vitals: Essential metrics for a healthy site,” Google LLC, 2026. [Trực tuyến]. Địa chỉ: https://web.dev/vitals/. [Truy cập: 25-09-2026].

[5] Tailwind Labs, “Tailwind CSS v4 Documentation and Oxide Engine Architecture,” 2026. [Trực tuyến]. Địa chỉ: https://tailwindcss.com/docs. [Truy cập: 22-09-2026].

[6] Microsoft Corp., “TypeScript Handbook: Strict Type Checking and Modern ECMAScript Modules,” 2026. [Trực tuyến]. Địa chỉ: https://www.typescriptlang.org/docs/. [Truy cập: 21-09-2026].

[7] D. Abramov, L. Tan, S. Markbåge, and J. Savona, “Introducing Zero-Bundle-Size React Server Components,” React, 2020. [Trực tuyến]. Địa chỉ: https://react.dev/blog/2020/12/21/data-fetching-with-react-server-components. [Truy cập: 24-09-2026].

[8] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, 2018. [Trực tuyến]. Địa chỉ: https://www.w3.org/TR/WCAG21/. [Truy cập: 26-09-2026].

[9] Node.js, “Domain Name System (DNS) Documentation,” OpenJS Foundation. [Trực tuyến]. Địa chỉ: https://nodejs.org/api/dns.html. Tài liệu module, không phải minh chứng phiên bản runtime của dự án.

[10] IEEE Computer Society, “IEEE Standard for Software User Documentation,” *IEEE Std 1063-2001 (R2007)*, pp. 1–45, 2007.

[11] Vercel, “Fetching Data,” Next.js Documentation. https://nextjs.org/docs/app/getting-started/fetching-data. Đối chiếu ngày 07-11-2026 với bản kèm Next.js

16.4.5: node_modules/next/dist/docs/01-app/01-getting-started/06-fetching-data.md.

[12] Vercel, “Server and Client Components,” Next.js Documentation. https://nextjs.org/docs/app/getting-started/server-and-client-components. Bản local: node_modules/next/dist/docs/01-app/01-getting-started/05-server-and-client-components.md. [Đối chiếu: 07-11-2026].

[13] Vercel, “fetch API Reference,” Next.js Documentation. https://nextjs.org/docs/app/api-reference/functions/fetch. Bản local: node_modules/next/dist/docs/01-app/03-api-reference/04-functions/fetch.md. [Đối chiếu: 07-11-2026].

[14] Vercel, “Caching without Cache Components,” Next.js Documentation. https://nextjs.org/docs/app/guides/caching-without-cache-components. Bản local: node_modules/next/dist/docs/01-app/02-guides/caching-without-cache-components.md. [Đối chiếu: 07-11-2026].

PHỤ LỤC

Phụ lục 1. Ranh giới Server/Client tại hàng phim

Ví dụ rút gọn từ `app/page.tsx`: Server Component tạo card; `MovieRow` phía client nhận phần tử đã tạo thay vì import file `MovieCard`. Cách truyền props/children này tuân theo hướng dẫn về interleaving của Next.js [1].

```
// app/page.tsx (Server Component)
<MovieRow
  title="Phim Thinh Hanh Trong Tuan"
  cards={trending.map((movie) => (
    <MovieCard key={movie.id} movie={movie} />
  ))}
/>
// components/client/MovieRow.tsx: "use client";
// Thanh cuon render phan tu da duoc truyen trong props.cards.
```

Phụ lục 2. Chọn Server Action hay Route Handler

Watchlist là thao tác ghi dữ liệu theo từng người dùng nên dùng `toggleWatchlist` trong `app/actions/watchlist.ts`; action kiểm tra `movieId`, ghi cookie và gọi `revalidatePath`. Live Search là thao tác đọc lặp lại khi nhập chữ nên dùng `GET /api/search?q=...`; client đợi 300ms và hủy request cũ bằng `AbortController`. Server kiểm tra từ khóa trước khi gọi TMDB. `s-maxage=120` là chỉ dẫn cho shared cache; mức độ cache thật cần kiểm tra trên môi trường triển khai.

Phụ lục 3. Quy trình kiểm chứng toàn sản phẩm

Yêu cầu Node.js 24 trở lên, dependency đã cài bằng `npm ci`, Internet tải font, quyền mở HTTP loopback và Chrome/Chromium. Đặt `CHROME_BIN` nếu cần, rồi chạy từ thư mục gốc:

```
npm run evidence
```

Lệnh tự chọn cổng loopback, chạy TMDB mock, lint/test, build/start Next production trong `.next-evidence`, typecheck, HTTP và Chrome, rồi dừng tiến trình đã tạo. Mock chỉ thay dịch vụ TMDB ngoài; Next Data Cache, streaming và Server Actions vẫn thực thi thật. Không cần khóa TMDB thật. TTL 15 giây chỉ được chấp nhận trên

mock HTTP loopback; không thay TTL của build thường. Mặc định lệnh ghi vào `baocao/evidence/`; chọn `MOVIEHUB_EVIDENCE_DIR` riêng nếu cần giữ nguyên bộ lịch sử. Bộ SP07 hiện được lưu riêng tại thư mục trong Bảng 4.1.

Đọc `mock-runtime.json` trong đúng thư mục của lượt vừa chạy; chỉ kết luận thành công khi `verification=full`, `status=passed` và exit code 0. Các artifact phải cùng `run_id` và `source_sha256`:

1. **Cache:** `cache-mock.json` có bảng `steps` ghi đầu vào, bộ đếm upstream và kết luận. ID mới tránh cache từ lúc build; chuỗi mỗi endpoint là $0 \to 1 \to 1 \to 2 \to 2$ qua miss, hit, hết TTL, refresh, hit mới. Client HTTP không prefetch; không suy số request origin từ số component hay thời gian phản hồi.
2. **Route:** `routes-http.json` xác nhận bốn hàng phim trong HTML server, query lặp chạy được, GET tìm kiếm có kết quả và ID phim sai có `noindex`.
3. **Streaming:** `streaming-mock.json` ghi chunk/phần chính/fallback/kết quả của nhánh credits và similar có độ trễ. Chỉ xác nhận thứ tự nhận HTML, không xác nhận thời điểm vẽ hoặc tương tác trên trình duyệt.
4. **Watchlist:** `watchlist-http.json` xác nhận thêm, tải lại bằng GET mới, xóa, tải lại vẫn trống và phiên cookie khác độc lập. Test đơn vị trực tiếp Server Action xác minh ID trùng, đầu vào sai, giới hạn 200 ID và 3800 byte.
5. **Chrome:** `browser-interactions.json` liệt kê 16 kiểm tra tương tác, bộ lọc, tìm kiếm, URL/Back/Forward, rollback, focus, responsive và carousel trong SP07. Không coi đây là điểm Lighthouse hay phép đo Web Vitals.

Hướng dẫn chụp bổ sung nằm tại `baocao/HUONG_DAN_THU_MINH_CHUNG.md`. Ảnh tĩnh lịch sử thuộc lượt riêng; SP07 có kiểm tra Chrome tự động nhưng chưa có Core Web Vitals, video hoặc Network waterfall. Quy trình học không cần Chrome nằm ở Phụ lục 5, dùng `npm run lab`.

Phụ lục 4. Khai báo sử dụng công cụ AI và minh chứng khắc phục lỗi

Hồ sơ trước kê khai sử dụng Copilot, Claude và ChatGPT; báo cáo chưa có hội thoại gốc để xác nhận từng đề xuất của các công cụ này. Phiên rà soát và ghép Hands-on Lab vào báo cáo dùng Codex để đối chiếu source, Git, metadata, chuyển nội dung lab sang LaTeX và chạy kiểm tra biên dịch. Kết quả lệnh tự động có thể đối chiếu; không mặc nhiên là bằng chứng thành viên đã tự kiểm tra độc lập hoặc hiểu toàn bộ mã.

Hai trường hợp dưới đây được hồ sơ cũ kê khai là lỗi liên quan AI. Commit

`953db35` xác minh thay đổi mã, không xác minh tác giả AI. Chúng được giữ như hồ sơ lịch sử với giới hạn nguồn, không trình bày như hai hội thoại AI đã được xác nhận:

1. Lỗi Monkey-Patching DNS toàn cục trong `services/client.ts`.

- **Nội dung hồ sơ khai:** Một đề xuất được quy cho AI đã ghi đè `dns.lookup` để đổi cách phân giải tên miền. Chưa có hội thoại gốc xác nhận nguồn đề xuất hoặc thử nghiệm mạng chứng minh nguyên nhân lỗi DNS.
- **Điều có thể kiểm tra:** Ghi đè hàm toàn cục ảnh hưởng các lời gọi dùng nó trong cùng tiến trình; không có dữ liệu để khẳng định đã xảy ra rò rỉ bộ nhớ. Commit `953db35` loại bỏ hook; mã hiện tại dùng DNS mặc định. Test kiến trúc trong SP07 kiểm tra không đưa hook trở lại.

2. Lỗi liên kết phim bộ tới route phim điện ảnh.

- **Nội dung hồ sơ khai:** Trang TV dùng card dẫn tới `/movie/[id]` dù ID lấy từ dữ liệu TV; nguồn đề xuất AI chưa được xác nhận bằng hội thoại gốc.
- **Điều có thể kiểm tra:** Hai loại tài nguyên dùng endpoint khác nhau, nên liên kết có thể trả 404 hoặc hiển thị phim điện ảnh khác khi ID trùng. Commit `953db35` loại bỏ trang `app/tv/page.tsx` và các lời gọi TV liên quan; bản bàn giao giới hạn ở phim điện ảnh. Không có lịch sử Git xác minh việc xóa một tệp `services/tv.ts`, cũng không có DB nội bộ được di trú trong thay đổi này.

Phiên cập nhật báo cáo này chỉ sửa tài liệu và PDF, không sửa source hay nhật ký riêng. Phần ai làm, thời điểm họp và xác nhận tự kiểm chứng của thành viên vẫn theo hồ sơ kê khai, chưa được xác minh đầy đủ; không tự điền sự kiện hoặc chữ ký. Hai lỗi đóng gói/hướng dẫn lab đã tái hiện và tình huống starter cố ý tạo được khai riêng trong Phụ lục 5, không dùng để dựng thêm lỗi AI.

Phụ lục 5. Hands-on Lab: Streaming và Data Cache

Dự án: MovieHub. **Đối tượng:** nhóm khác đã biết React, TypeScript và `async/await`. **Thời lượng dự kiến:** 40 phút thực hành, không tính chuẩn bị môi trường. Phần này ghép toàn bộ nội dung học vào cuối báo cáo; bộ tài liệu thao tác độc lập vẫn nộp kèm tại `labs/streaming-cache/`, gồm `README.md`, `starter.patch`, `WORKSHEET.md`, `SOLUTION.md` và minh chứng. Người học đọc đáp án sau khi hoàn thành ba bài.

5.1. Lý do chọn chủ đề, mục tiêu và phần lõi — D0

MovieHub lấy thông tin phim, diễn viên và phim tương tự từ TMDB trên server. Nếu trang cha chờ toàn bộ dữ liệu, người xem phải đợi phần chậm nhất dù các phần phụ đã có Suspense. Nếu không hiểu Data Cache, người phát triển cũng dễ coi dữ liệu cũ sau TTL là lỗi hoặc gọi mọi cơ chế cache là ISR.

Lab tập trung hai kỹ thuật đã dùng trong sản phẩm: **Streaming với Server Components** và **Data Cache có revalidation**. Người học sửa vị trí chờ dữ liệu, kiểm tra HTML và số request thực tế; không dành buổi học để dựng CRUD hay chép giao diện. Sau lab, người học có thể:

1. Vẽ luồng dữ liệu trang phim, xác định Server/Client boundary và nơi giữ API key.
2. Sửa việc chờ dữ liệu phụ làm chặn nội dung chính; chứng minh fallback và kết quả đến ở các thời điểm khác nhau.
3. Đối chiếu miss/hit/refresh bằng counter và nội dung thay đổi; phân biệt Data Cache, memoization và cache trình duyệt.

Theo `REQUIRE.md`, Bài 1 chạm ba mục Tầng 1A bắt buộc: phân biệt Server/Client Components, fetch phía server và hiểu luồng render chính. Bài 2–3 đào sâu Suspense/Streaming và caching/revalidation của Tầng 1B. Lab không dạy toàn bộ tổ chức page/layout và cây App Router. Phân bổ dự kiến: Bài 1 (7 phút), Bài 2 (18 phút), Bài 3 (10 phút), hoàn thiện bài nộp (5 phút).

5.2. Chuẩn bị trước buổi học

- Node.js từ 24, npm, Git và quyền chạy HTTP trên `127.0.0.1`.
- Source MovieHub đi kèm lab, giữ thư mục `.git`; không cần tài khoản hoặc khóa TMDB thật.

- Internet để cài dependency và tải Google Fonts khi build. Mock không làm build hoàn toàn offline.
- Bash hoặc PowerShell; lệnh thực hành bên dưới dùng chung cho hai shell. Không cần Chrome hoặc ba terminal. Các lệnh mới được kiểm chứng trực tiếp trên Linux/Bash, chưa kiểm chứng PowerShell/Windows.
- Next.js `16.4.5`, React `19.2.8`, `cacheComponents: false`; giữ `package-lock.json`.

Làm trên **bản sao riêng** của source; bỏ `node_modules`, `.next`, `.next-evidence` khi sao chép, giữ `.git` và thư mục lab. Nếu tài liệu chưa commit, clone không mang theo thay đổi chưa commit. Mở terminal ở gốc bản sao và chạy trước buổi học:

```
node --version
git --version
npm ci
git rev-parse HEAD
git diff HEAD -- "app/movie/[id]/page.tsx"
```

Diff cuối phải trống. Nếu trang phim có sửa riêng, giữ sửa đổi đó và dùng bản sao sạch khác; không dùng `git reset --hard`. Khắc phục lỗi cài dependency hoặc tải font trước khi tính thời gian thực hành. Không cần migration, seed DB hay model/index theo Mục 3.1.1..

5.3. Bài 1 — Đọc luồng và dự đoán (7 phút)

Bước 1. Tạo trạng thái starter và lưu mốc so sánh.

```
git apply --check labs/streaming-cache/starter.patch
git apply labs/streaming-cache/starter.patch
node -e "const fs = require('node:fs'); fs.mkdirSync('labs/streaming-
    cache/work', { recursive: true }); fs.copyFileSync('app/movie/[id]/
    page.tsx', 'labs/streaming-cache/work/page.starter.txt', fs.constants.
    COPYFILE_EXCL)"
```

Nếu `--check` thất bại, dừng và xem mục lỗi thường gặp; không ép áp dụng. Nếu snapshot báo `EEXIST`, giữ bản cũ và dùng bản sao mới cho lượt khác. Snapshot `.txt` không bị TypeScript coi là trang mới.

Starter thêm hai lời gọi dữ liệu phụ vào `Promise.all` trước `return`. Đây là tình huống chặn **cố ý để học**, không phải lỗi được phát hiện trong sản phẩm gốc. Nội dung patch đi kèm được hiển thị nguyên tệp dưới đây; áp dụng tệp source, không cần gõ lại từ PDF:

```
--- a/app/movie/[id]/page.tsx
+++ b/app/movie/[id]/page.tsx
@@ -3,7 +3,7 @@
 import Link from "next/link";
 import { notFound } from "next/navigation";
 import type { Metadata } from "next";
-import { getMovieDetails, getMovieVideos, selectBestTrailer, TMDBError } from "@/services/
    tmdb";
+import { getMovieDetails, getMovieVideos, selectBestTrailer, TMDBError, getMovieCredits,
    getSimilarMovies } from "@/services/tmdb";
 import { formatRuntime } from "@/utils/formatRuntime";
 import CastList from "@/components/server/CastList";
 import CastSkeleton from "@/components/server/CastSkeleton";
@@ -94,6 +94,9 @@
   const [isSaved, videos] = await Promise.all([
     isMovieSaved(movie.id),
     getMovieVideos(movie.id.toString()),
+    // Lab starter: ữd ệliu ụph ịb ờch ướtrc khi render các ranh ógii Suspense.
+    getMovieCredits(id),
+    getSimilarMovies(id),
   ]);

   const bestTrailer = selectBestTrailer(videos);
```

Bước 2. Đọc các phần liên quan.

Bảng PL5.1. Các file cần đọc trong Bài 1

File	Điều cần xác định
`app/movie/[id]/page.tsx`	Dữ liệu cần trước `return`, hai Suspense và dữ liệu bị chờ thêm.
`components/server/CastList.tsx`, `components/server/SimilarMovies.tsx`	Nơi fetch dữ liệu phụ, xử lý lỗi và dựng nội dung.
`components/client/CastSlider.tsx`, `components/client/SimilarMoviesSlider.tsx`	Tương tác chạy ở trình duyệt.
`services/client.ts`	API key, timeout, `next.revalidate` và giới hạn TTL mock.

Bước 3. Điền Bài 1 trong phiếu ở mục 5.7. Vẽ luồng request, đánh dấu nơi giữ khóa và dự đoán vì sao hai Suspense chưa giúp nội dung chính đến sớm. Chưa xem đáp án.

Đạt khi: phân biệt được nơi fetch và nơi tương tác; chỉ ra `await` ở cha diễn ra trước khi hai Suspense được render.

5.4. Bài 2 — Sửa và kiểm chứng streaming (18 phút)

Bước 4. Chạy trên starter.

```
npm run lab
```

Lệnh tự chạy lint/test, mock TMDB, build production, typecheck và HTTP; tự chọn cổng và dừng server khi xong. Dòng `Evidence directory:` in thư mục kết quả ngay đầu lượt chạy. Mỗi lượt nằm riêng dưới `labs/streaming-cache/evidence/runs/`; ghi lại đường dẫn.

Starter phải kết thúc với exit code **1** vì streaming, ví dụ:

```
cast_result must arrive progressively, at least 1000ms after main HTML in
    this mock
```

Xem exit code ngay sau lệnh: Bash dùng `echo $?`, PowerShell dùng `$LASTEXITCODE`. Mở `mock-runtime.json`: phải có `verification=lab`, `status=failed` và `error` thuộc streaming. Log lint, test, build và typecheck và `routes-http.json` phải cho thấy các bước trước đó đạt. Lỗi tải font, kết nối hoặc build **không** phải thất bại mong đợi. Starter dừng trước bước cache nên chưa có `cache-mock.json` là bình thường.

Bước 5. Sửa `app/movie/[id]/page.tsx`.

1. Bỏ hai lời gọi dữ liệu phụ đang bị chờ trong `Promise.all` trước `return`, dọn hai import thừa.
2. Giữ fetch trong `CastList` và `SimilarMovies`, dưới hai Suspense riêng.
3. Giữ validation ID, xử lý lỗi, skeleton, `data-stream`, `data-movie-id` và Server/Client boundary. Không chuyển trang sang Client Component.

Bước 6. Chạy lại.

```
npm run lab
```

Lệnh build lại mã đã sửa, dùng ID mới và tạo thư mục kết quả mới. Cần đạt:

- Exit code 0; `mock-runtime.json` có `verification=lab`, `status=passed`.
- `streaming-mock.json` có đủ `main`, `cast_fallback`, `similar_fallback`, `cast_result`, `similar_result` trong `events`.
- Nội dung chính/fallback đến ở chunk trước kết quả phụ; mỗi kết quả phụ cách `main` ít nhất 1.000 ms theo điều kiện mock này.

Ghi số liệu của **cùng lượt solution** vào phiếu. Không yêu cầu số chunk hoặc thời gian tuyệt đối giống mẫu.

Bước 7. Tạo diff so với snapshot starter.

```
git diff --no-index -- labs/streaming-cache/work/page.starter.txt "app/movie/[id]/page.tsx"
```

Lưu diff vào bài nộp. Với lệnh này, exit code **1 nghĩa là có khác biệt**, 0 là không khác biệt, lớn hơn 1 là lỗi. Snapshot cần thiết vì sửa đúng có thể trở về nguyên bản HEAD, khiến `git diff` thông thường trống.

Đạt khi: có lỗi streaming trước sửa, kết quả đạt sau sửa và diff giải thích được. Giữ comment starter cũng hợp lệ nếu không còn hai lời gọi chặn.

5.5. Bài 3 — Đối chiếu Data Cache (10 phút)

Bước 8. Dự đoán trước khi đọc kết quả. Cho một endpoint, dự đoán counter ở năm mốc: trước tải, cold miss, warm hit, refresh sau TTL và hit sau refresh. Điền dự đoán vào phiếu.

Bước 9. Đối chiếu cùng lượt solution vừa chạy. Mở `cache-mock.json`, kiểm tra bốn endpoint của cùng ID:

- `/movie/<id>` và các suffix `/videos`, `/credits`, `/similar`.
- Năm trường `before`, `miss`, `hit`, `after_expiry`, `warm_after_refresh`.
- `content_before`, `content_after`, `ttl_seconds`, `cache_components` và mô tả request trong `steps`.

Điền counter, nội dung trước/sau và dự đoán nào cần sửa. Script thực hiện request HTTP thật tới Next production; mock origin tăng counter, chờ TTL và đổi dữ liệu. Đây là kết quả lần chạy của người học, không phải bảng số dựng sẵn.

Bước 10. Trả lời.

1. Request đầu sau TTL có bắt buộc nhận ngay dữ liệu mới không? Vì sao? Hết TTL có tự sinh request khi không có người truy cập không?
2. Counter tăng ở `START` hay `END`? Điều gì chứng minh refresh đã sẵn sàng?
3. Vì sao không quy kết kết quả cho cache trình duyệt hoặc memoization? `AbortSignal` trong helper ảnh hưởng gì?

4. Route đọc cookie Watchlist nhưng fetch TMDB có cache: nên mô tả mô hình render/cache thế nào? Có thể suy tốc độ TMDB thật, Web Vitals hoặc cache CDN từ phép đo không?

Đạt khi: đối chiếu đủ bốn endpoint, xác nhận nội dung thay đổi và giải thích stale-while-revalidate. Không suy hiệu năng TMDB thật từ mock.

5.6. Nộp bài (5 phút)

Nộp phiếu đã điền, sơ đồ, diff so với starter và hai thư mục kết quả của chính nhóm thực hành:

- **Starter:** `mock-runtime.json` báo lỗi streaming; log lint, test, build và typecheck và `routes-http.json` cho thấy các bước trước đó đạt.
- **Solution:** `mock-runtime.json`, `streaming-mock.json`, `cache-mock.json` cùng log lint, test, build và typecheck.
- Mỗi bộ phải cùng `run_id` và `source_sha256`. Starter và solution có trạng thái nguồn khác nhau; không ghép artifact của chúng.

Khai phần tự làm và phần có AI hỗ trợ. Không nộp khóa thật, cookie hoặc `.env.local`. Các ô trống dưới đây dành cho người học, không phải dữ liệu còn thiếu của nhóm biên soạn.

5.7. Phiếu thực hành dành cho người học

Điền bằng kết quả của chính nhóm; không chép số liệu mẫu thành lần chạy của mình. Có thể điền vào `WORKSHEET.md` đi kèm thay vì viết trực tiếp trên PDF.

Bảng PL5.2. Thông tin nhóm thực hành — dành để người học điền

Mục	Kết quả của người học
Nhóm, thành viên, lớp	
Ngày chạy, hệ điều hành	
Phiên bản Node.js / npm / Next.js	
Git HEAD, thay đổi trong working tree	
Thời gian bắt đầu/kết thúc, không tính cài dependency	
Công cụ AI, phần hỗ trợ và phần tự kiểm tra	

Phiếu Bài 1. Vẽ luồng request tới trang phim, helper fetch, hai component phụ và component tương tác. Đánh dấu Server/Client boundary và nơi giữ API key. Đính kèm sơ đồ của nhóm, rồi trả lời:

1. Dữ liệu nào cần trước `return` để dựng thông tin phim chính?
2. Hai lời gọi nào của starter không cần chặn phần chính? Vì sao hai Suspense chưa giúp được?
3. Có nên chuyển trang sang Client Component hoặc đưa khóa vào `NEXT_PUBLIC_` để sửa không? Giải thích.

Phiếu Bài 2. Điền hai lượt riêng và đính kèm diff so với `work/page.starter.txt`. Giải thích bản sửa và vì sao `Promise.all` vẫn có thể chặn nội dung chính.

Bảng PL5.3. Đối chiếu trước/sau sửa — người học điền

Mục	Starter	Solution
Thư mục kết quả được in		
`run_id`		
`source_sha256`		
`verification`, `status`, exit code		
Kết quả lint, test, build, typecheck và route		
Nguyên văn lỗi nếu có		

Bảng PL5.4. Marker của một lượt solution — người học điền

Marker	Nhận sau (ms)	Chunk
`main`		
`cast_fallback`		
`similar_fallback`		
`cast_result`		
`similar_result`		

Tính khoảng cách từ `main` đến mỗi kết quả phụ. Vì sao nhiều chunk chưa đủ chứng minh streaming hữu ích? Các số đo này có chứng minh thời điểm trình duyệt vẽ không?

Phiếu Bài 3. Trước khi đọc kết quả, ghi dự đoán counter của một endpoint qua năm mốc. Ghi ID đo, TTL và `cache_components`. Điền bảng từ cùng một `cache-mock.json`; các cột lần lượt tương ứng `before`, `miss`, `hit`, `after_expiry`, `warm_after_refresh`.

Bảng PL5.5. Counter cùng một ID — người học điền

Endpoint	Trước	Miss	Hit	Refresh	Hit mới
`/movie/<id>`					
`/movie/<id>/videos`					
`/movie/<id>/credits`					
`/movie/<id>/similar`					

Bảng PL5.6. Nội dung sau refresh — người học điền

Kiểm tra	Kết quả
`content_before`	
`content_after`	
Nguồn chứng minh refresh xong: assertion nội dung hoặc log `END`	
Dự đoán nào cần sửa, vì sao	

Trả lời bốn câu hỏi ở Bước 10, bao gồm phạm vi cache và giới hạn phép đo.

Checklist trước khi nộp.

- ☐ Ba bài đã điền, có sơ đồ và diff so với snapshot starter.
- ☐ Starter thất bại đúng tại streaming, có lỗi và exit code; các bước trước đó đạt.
- ☐ Solution có `verification=lab`, `status=passed`, exit code 0.
- ☐ Có streaming/cache và log lint, test, build và typecheck của solution.
- ☐ Mỗi lượt khớp `run_id`, `source_sha256`; hai trạng thái nguồn tách riêng.
- ☐ Đã nêu Next production chạy thật, TMDB dùng mock loopback và giới hạn đo.
- ☐ Đã khai AI hỗ trợ nếu có; không nộp khóa thật, cookie hoặc `.env.local`.

5.8. Lỗi thường gặp và kinh nghiệm đã kiểm chứng

Hai lỗi sau đã được tái hiện khi hoàn thiện bản lab trước. Đây là lỗi công cụ/hướng dẫn lab, không phải bằng chứng AI trả lời sai:

Bảng PL5.7. Hai lỗi của bản hướng dẫn trước và cách xử lý

Lỗi đã gặp	Nguyên nhân	Cách bản mới xử lý
Sửa đúng nhưng test patch thất bại khi giữ comment	Test coi khớp patch là điều kiện đúng của lời giải.	Tách test patch thành kiểm tra đóng gói; nghiệm thu bằng streaming/cache.
Sửa xong nhưng `git diff` trống	Source bàn giao vốn đã có lời giải.	Lưu snapshot starter rồi dùng `git diff --no-index`.

`rebuilt-checks/checks.json` ghi lời giải giữ comment được chấp nhận, diff có nội dung và patch roundtrip. Các tình huống dưới đây là hướng dẫn xử lý, không mặc định đều là sự cố nhóm từng gặp:

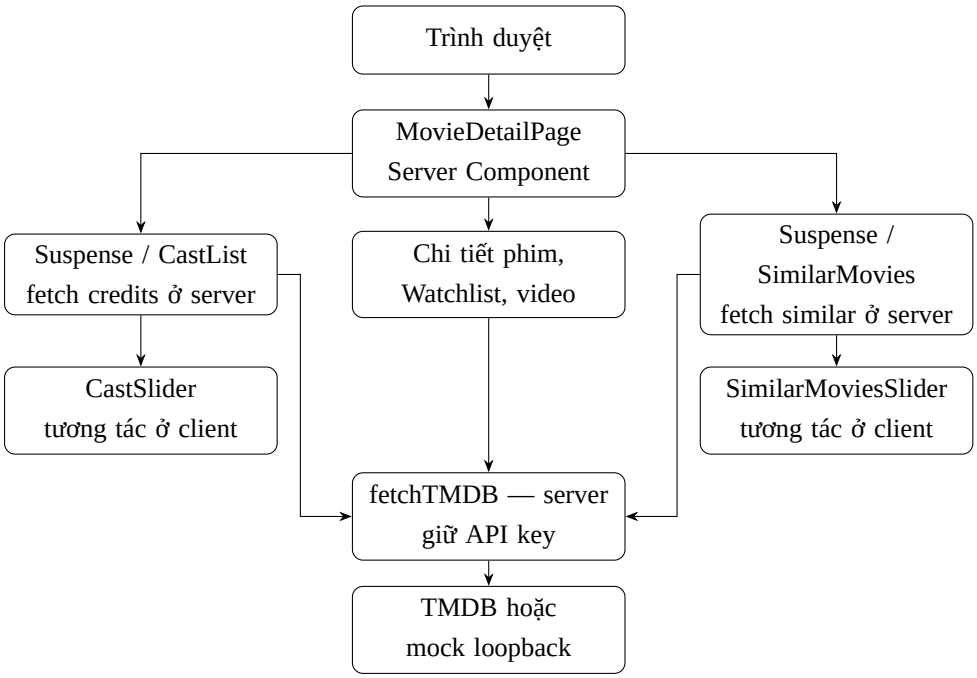
Bảng PL5.8. Chẩn đoán khi người học gặp trở ngại

Triệu chứng	Kiểm tra và xử lý
Patch không áp dụng	Kiểm tra đúng gốc project, starter đã áp dụng chưa, trang có sửa riêng không. Dùng bản sao sạch; không ép patch.
`EEXIST` khi lưu snapshot	Giữ snapshot cũ; dùng bản sao mới cho lượt khác.
Build không tải được Google Fonts	Kiểm tra HTTPS/Internet rồi chạy lại; không tính là lỗi streaming.
`Local HTTP is blocked by the sandbox` hoặc lỗi quyền mở server	Cấp quyền chạy loopback trong môi trường của người học rồi chạy lại.
Streaming không đạt sau sửa	Kiểm tra đúng file, hai lời gọi ở cha đã bỏ chưa; chạy lại `npm run lab` để build mới.
Đọc nhầm starter/solution	Đối chiếu đường dẫn, `run_id`, fingerprint và `status`; không lấy số từ thư mục lịch sử.

Chưa có log đủ để khẳng định nhóm đã gặp mọi lỗi chuyên môn trong bảng này. Tình huống chặn của starter được tạo có chủ đích và khai riêng.

5.9. Đáp án — đọc sau khi làm ba bài

Bài 1: Luồng và boundary. Trang cần chi tiết phim, trạng thái Watchlist và video để dựng phần chính. Cast/similar có thể hoàn tất sau dưới hai Suspense. Slider, nút Watchlist và modal xử lý tương tác ở client; fetch TMDB nằm phía server. API key được đọc trong `services/client.ts`, không đưa vào `NEXT_PUBLIC_`, props hoặc mã trình duyệt. Khóa `evidence-only` trong lab là giả; `HttpOnly` của cookie Watchlist không liên quan tới việc giữ khóa TMDB trên server.



Hình PL5.1. Đáp án luồng dữ liệu trang phim; các mũi tên mô tả quan hệ gọi/truyền dữ liệu

Starter chờ credits/similar trong `Promise.all` **trước khi trả JSX**. Hai Suspense bên dưới chưa được render nên chưa giúp nội dung chính đến sớm. `loading.tsx` cấp route có thể xuất hiện nhưng không chứng minh thông tin phim chính đã đến. Không cần chuyển trang sang client hoặc công khai khóa để sửa.

Bài 2: Bản sửa tối thiểu. Trong `app/movie/[id]/page.tsx`, bỏ hai import và hai lời gọi của starter. Đoạn sau minh họa trạng thái sau sửa; không thay toàn bộ trang bằng đoạn rút gọn này:

```
import { getMovieDetails, getMovieVideos, selectBestTrailer, TMDBError }
    from "@/services/tmdb";

const [isSaved, videos] = await Promise.all([
  isMovieSaved(movie.id),
  getMovieVideos(movie.id.toString()),
]);
```

Giữ fetch của `CastList` và `SimilarMovies` trong các Suspense riêng. Có thể giữ comment starter mà vẫn đúng hành vi; không thêm fetch trình duyệt, xóa skeleton hay đổi boundary.

`Promise.all` khởi động tác vụ song song nhưng `await` vẫn chờ tác vụ chậm nhất. Sau sửa, phần chính/fallback được gửi trước kết quả phụ. Verifier yêu cầu năm marker HTML thật, phần chính/fallback ở chunk trước kết quả và mỗi kết quả phụ cách `main` ít nhất 1.000 ms trong điều kiện mock. Response bị buffer vẫn có thể tách thành nhiều chunk transport cùng lúc; chỉ đếm chunk là chưa đủ. Thời điểm HTTP client nhận HTML không phải lúc trình duyệt vẽ hay hydration.

Starter đúng bài khi lint, test, build, typecheck và route đạt rồi thất bại tại streaming. Solution đúng khi lượt mới có `verification=lab`, `status=passed`, exit code 0. Diff nộp so với snapshot starter; so với HEAD có thể trống. Exit code 1 của `git diff --no-index` là có khác biệt, không phải lab thất bại.

Bài 3: Counter, nội dung và phạm vi cache. Kỳ vọng cho từng endpoint phim và ba suffix:

Bảng PL5.9. Đáp án vòng đời Data Cache trong điều kiện mock

Mốc	Counter	Nội dung
`before`, ID mới	0	Chưa tải.
`miss`, cold request đã đọc hết	1	Phiên bản 1.
`hit`, trong TTL	1	Phiên bản 1.
`after_expiry`, đã kích hoạt refresh	2	Request kích hoạt có thể còn nhận bản cũ.
`warm_after_refresh`, refresh hoàn tất	2	Phiên bản 2.

1. **Sau TTL:** request đầu có thể nhận dữ liệu stale và kích hoạt revalidation nền. Hết TTL không tự sinh request khi không có người truy cập; không khẳng định request kích hoạt bắt buộc trả Phiên bản 2.

2. **Counter và hoàn tất:** mock tăng counter tại `START`. Counter tăng chưa chứng minh response đã xong. Verifier đợi refresh rồi xác nhận HTML chứa Phiên bản 2; có thể đối chiếu `END` trong log. Warm hit không làm counter tăng tiếp.
3. **Ba loại ghi nhớ:** Data Cache giữ response giữa các request tới Next; request memoization có phạm vi một lượt render; cache trình duyệt ở phía người dùng. Bộ đo dùng HTTP client Node với request mới. Helper truyền `AbortSignal.timeout(10000)`, opt out memoization fetch mặc định; không quy counter ở đây cho memoization. Page và `generateMetadata` cùng gọi helper không tự chứng minh origin nhận hai request; counter mới là bằng chứng.
4. **Mô hình route:** đọc cookie Watchlist làm trang có dữ liệu theo request, trong khi fetch TMDB có `next.revalidate` vẫn dùng Data Cache. Mô tả là **SSR kết hợp Data Cache**, không suy ISR toàn trang; lab giữ `cacheComponents: false`. Counter không phải thời gian phản hồi; mock cục bộ không chứng minh tốc độ TMDB thật, Web Vitals hoặc cache CDN.

5.10. Minh chứng và khả năng chạy lại của bộ lab

Các bộ LB07, ST07 và CM07 trong Bảng 4.1 chạy ngày 07/11/2026 trên Linux với Node `v26.12.1`, Next `16.4.5`; metadata giữ nguyên HEAD `7e06a9a` và working tree có thay đổi. Hai lượt canonical LB07/SP07 cùng fingerprint nhưng khác `run_id` và phạm vi. ST07/CM07 có fingerprint khác vì mã trang khác.

Số liệu sau chỉ trích từ `streaming-mock.json` thuộc **LB07**, không ghép với SP07:

Bảng PL5.10. Streaming mẫu của lượt LB07 — không thay kết quả người học

Marker	Nhận sau (ms)	Chunk
`main`	58	22
`cast_fallback`	59	23
`similar_fallback`	59	23
`cast_result`	2564	29
`similar_result`	4073	55

`cache-mock.json` của cùng LB07 dùng ID `1794042737842`. Cả bốn endpoint phim, videos, credits và similar đều có chuỗi counter $0, 1, 1, 2, 2$, nội dung

được kiểm tra đổi từ **Phiên bản 1** sang **Phiên bản 2**. Credits chậm 2,5 giây, similar chậm 4 giây và TTL mock 15 giây; TTL dịch vụ thật giữ nguyên.

Kiểm tra đóng gói dành cho nhóm biên soạn trên source bàn giao:

```
node --test labs/streaming-cache/check-starter.test.mjs
```

Kiểm tra áp dụng/đảo ngược patch trên bản sao rồi so khớp từng byte. Đây là một ca riêng, không nằm trong `npm test` hoặc điều kiện chấm lời giải; bộ chính có 43 ca. Học viên dùng `npm run lab` để nghiệm thu hành vi. `npm run evidence` vẫn kiểm tra toàn sản phẩm với Watchlist/Chrome; lab không cần Chrome. Giữ thư mục `evidence/runs/` riêng mỗi lượt để không ghi đè.

`labs/streaming-cache/evidence/rebuilt-checks/checks.json` ghi cài dependency trong checkout mới, starter/snapshot, lời giải giữ comment, diff và patch roundtrip. `install-output.txt` và `solution-diff.txt` cùng thư mục là log đối chiếu. Checkout mới cài dependency mất 7,927 giây; lệnh starter mất 41,097 giây, solution giữ comment mất 55,298 giây. Đây là thời gian lệnh tự động, **không phải thời gian nhóm sinh viên tự học**. Mốc 40 phút chưa được xác nhận bằng buổi học với nhóm khác. Chưa kiểm chứng PowerShell/Windows hoặc một lượt CI GitHub.

Các thư mục lab `evidence/starter/`, `evidence/solution/` và tệp install/manual/patch cũ thuộc bản lab 75 phút cùng fingerprint trước lần viết lại; giữ để truy lịch sử, không ghép với bộ mới hoặc dùng con số 44 test cũ đại diện bản hiện tại.

5.11. Đối chiếu yêu cầu và nguồn học

- **D0:** mục 5.1 giải thích chủ đề cốt lõi/kỹ thuật đã làm, lý do chọn và năng lực sau học.
- **Năm nội dung tối thiểu:** mục tiêu tại 5.1; môi trường tại 5.2; starter và bước đánh số tại 5.3–5.5; ba bài cùng phiếu tại 5.7 và đáp án tại 5.9; lỗi đã gặp và cách xử lý tại 5.8.
- **Tài liệu nộp kèm:** bộ lab riêng giữ trong source; bản ghép này nằm ở cuối báo cáo theo yêu cầu. Ảnh chỉ cung cấp chi tiết điểm D0, không tự suy thang điểm D1–D3.

Nguồn học: Suspense [11]; Server/Client Components [12]; fetch/revalidation và memoization [13]; Data Cache [14].